

CS3217: Software Engineering on Modern Application Platforms

AY 2023/2024, Semester 2

Final Report

TowerForge

Name	Matric Number
Keith Gan	[redacted]
Phua Zheng Ze	[redacted]
Rubesh Suresh	[redacted]
Vanessa Mae	[redacted]

Table of Contents

1 – Project Overview....4
2 – Features and Specifications....5
Game Features....5
App Features....6
3 – User Manual....7
Objective....7
Combat rules....7
Game Modes....7
Capture the Flag....8
Death Match....8
Survival Mode....9
Multiplayer....9
User Registration and Authentication....10
Leaderboard....10
Player Screen & Progress Saving....10
4 – TowerForge Design Overview....13
Top Level Design....13
Entity Component System....14
Events API....15
Rendering API....19
MetricsAPI....21
StatisticsAPI....21
InferenceAPI....22
MetricsAPI and Single Responsibility Principle....22
StorageAPI....23
Usage of Libraries....25
Design Problems....25
Network API....25
Event Transformations....25
5 – Runtime Structure....26
Relevant Classes....26
GameWorld....26
EntityManager....26
SystemManager....26
Implementation of CheckRepresentation....26
Overview....26
Runtime Vulnerability....26
CheckRepresentation Function....27
Usage....27
Entity Manager....27
6 – Module Structure....28
Module Structure and Diagrams....28
ECS Framework....29
Game Modes....30

Game Room....32
Game Network....32
Authentication Manager....33
Leaderboard Provider + Game Ranking....34
Power Ups....35
Invulnerability Power Up....36
Damage Power Up....36
No Cost Power Up....36
Future Power Ups....36
7 – Reflection....39
Lessons....39
Evaluation....39
Overall Reflection....40
Known Bugs and Unresolved Issues....41
Networking Latency....41
Lag Propagation....41
8 – Appendix....42
A – GUI....42
B – Test Cases....43
Unit Tests....45
GameMode....45
GameModeFactory....45
GameRoom....45
Authentication Tests....46
Game Ranking + LeaderBoard Tests....46
Integration Testing....47
Authentication Integration Tests....47
LeaderBoard Integration Tests....47
Game Room Integration Tests....47
Game Network Integration Tests....49
Storage + Sync Tests....50
UI Testing....51
Authentication UI Tests....51
C – Project Schedule....52
Sprint Objectives....52
D – Individual roles and contribution....53

CS3217 Final Report: TowerForge



1 – Project Overview

Preface

TowerForge is a multiplayer strategic tower defense game set in a fictional world where players must defend their fortress from waves of enemy invaders. Players strategically place various towers along the enemy's path to destroy them before they breach the fortress gates and lose. As the game progresses, players earn game points to place more fortresses or summon more units and unlock to fend off increasingly challenging waves of enemies.

Overview

TowerForge is a multiplayer grid-based tower defense builder that offers a variety of game modes, units, and power-ups for players to enjoy. Players can choose different game modes before diving into the action. In TowerForge, players must strategically place units (minions) to attack their opponent's fortress, reducing their Health Points (HP). The player whose fortress HP reaches zero first loses the game.

Gameplay

In TowerForge, players take on the role of a commander defending their fortress from enemy incursions. As the game progresses, players accumulate points and achievements, rewarding their strategic prowess and successful defenses. These achievements not only provide a sense of accomplishment but also unlock new towers, units, and power-ups, allowing for further customization of defenses.

Throughout the game, players must adapt to increasingly formidable waves of enemies, managing resources and unlocking new towers and units to stay ahead. TowerForge offers a dynamic and challenging experience, with achievements serving as milestones of progress and skill.

2 – Features and Specifications

Game Features

MeleeUnit	- A movable unit that deals damage on contact with units of opposing team - Moves faster than soldier unit - Moves towards the enemy base. Stops when in contact with an enemy entity. - Takes damage from damage causing entities from opposing team - Disappears when their health reaches 0
SoldierUnit	- A movable unit that is able to spawn Bullets to deal damage to entities of the opposing team. - Takes damage from damage causing entities from opposing team - Moves towards the enemy base. Stops when in contact with an enemy entity. - Disappears when their health reaches 0
WizardUnit	- A movable unit that is able to spawn Magic Balls to deal damage to entities of the opposing team. - Moves much faster than other units - Moves towards the enemy base. Stops when in contact with an enemy entity. - Takes damage from damage causing entities from opposing team - Disappears when their health reaches 0
ArrowTower	- An immovable unit that can be placed on the map to protect a lane. - Able to spawn Bullets to to deal damage to entities of the opposing team - Takes damage from damage causing entities from opposing team - Disappears when their health reaches 0
Bullet/Wizard Ball	- A movable projectile that is able to deal damage with entities of the opposing team. - Disappears when dealing damage to an entity of the opposing team
Team	- Represents the player. - Contains the number of lives left by the player. This represents the number of entities that needs to be let into the base for the player to lose - Contains and generates the points that are used to purchase units.
AiComponent	- Acts as the player 2 in single player mode. - Randomly chooses a random unit and spawns it on a random tile on the board provided that it has enough points to spawn the unit. - A random delay is added before it attempts to spawn another unit.
Power Ups	- Players have access to multiple power ups including: - Invulnerability - Increase Damage - No Unit Costs

App Features

Game Modes	- Players will have access to 3 different game modes. They are: - Capture The Flag - Death Match - Survival
Multiplayer	- Players are able to play both in single player against the enemy AI or against anyone in the world. - All changes to game state will be synchronised between players and each player can see the actions that the other player plays
Achievements	- Players can earn achievements as they play the game. These achievements include number of kills, number of games and more
Leaderboard	- Players will have access to the global rankings where they can compete with each other in different metrics.
Player Screen	- Players can have a detailed stat breakdown of their time in the game, earning points which are then represented as ranks in the game. Players can compete to see who manages to receive the highest rank.
Player Progress Saving	- Players can carry over their progress to another device simply by logging in to that other device. Player progress will be preserved on both devices.

3 – User Manual

Welcome to TowerForge!

Objective

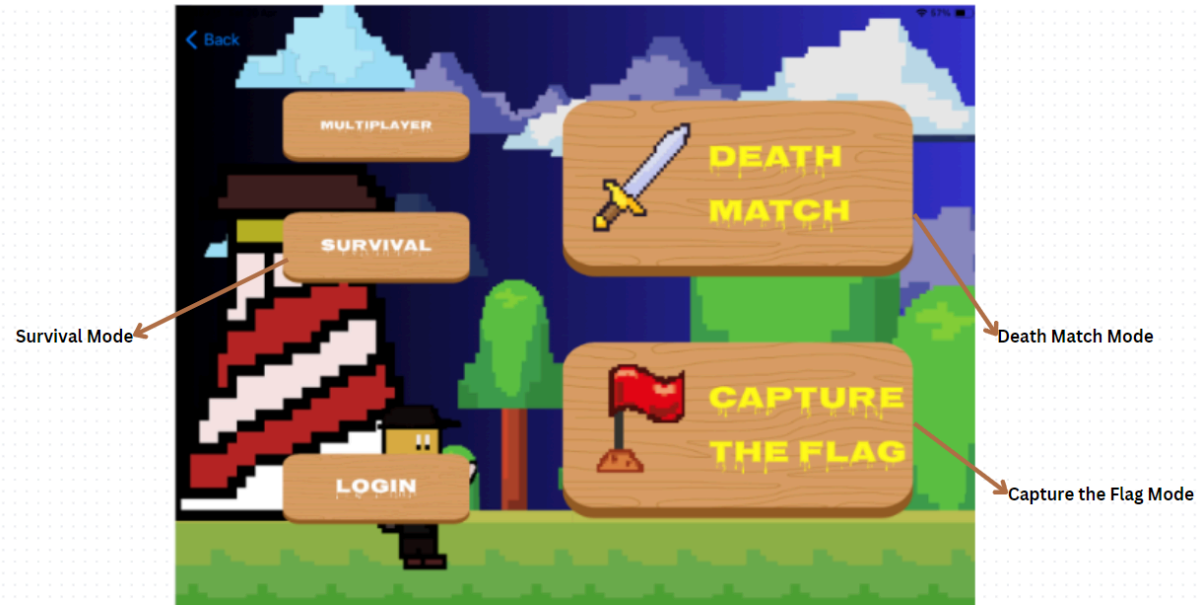
TowerForge is a tower defense game. Your objective is to build an army that can defeat your opponents before the time runs out. In this game you have three base units: Melee, Soldier, and Arrow Tower.

Combat rules

A Melee unit will only begin attacking when it contacts an enemy unit. A Soldier unit will begin firing as soon as an enemy unit is within its firing range. Lastly, an arrow tower is a static defense unit which is able to block enemy units while firing projectiles at incoming enemies.

Game Modes

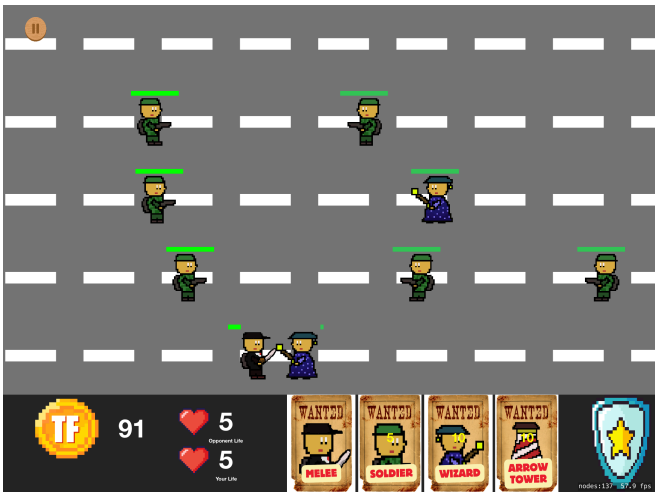
TowerForge offers diverse gameplay modes with different game mechanics and objectives. Currently, it supports the “DeathMatch” mode, “Survival” mode and the “Capture the Flag” mode that users can choose at the beginning in the game mode selection screen as shown below.



Capture the Flag

Objective: Capture the opponent's base while defending their own base. Each player will have their own life points for their base and attack the opponent by placing units so that it reaches the opponent's base. Upon successful infiltration of the unit into the opponent's base, the life point of the opponent's base will be reduced.

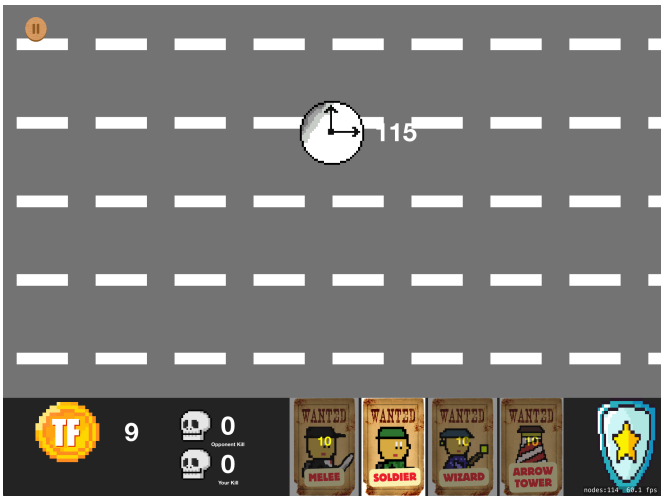
Winning condition: The player with zero life point left will lose and the opposite team will win



Death Match

Objective: Eliminate as many opponent's units within the time limit. Eliminating a unit will earn the player points and hence this would contribute to their winning points.

Winning condition: The player with the most point at the end of the game will win



Survival Mode

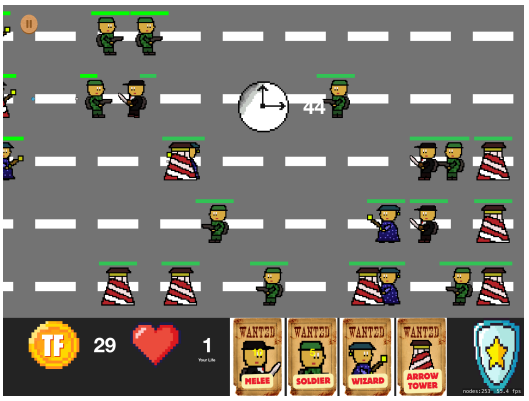
Objective: There will be a time limit that the player needs to survive until the time ends. Every 50 seconds, there will be a new wave of units on the opponent's side. Players can choose what level they want to play the game in where each level means more waves and higher waves means higher difficulty as well. There will be a sound effect played when a wave is done.

Winning condition: The player needs to survive until the game ends with only 1 life unit.

Choosing Level (Currently 3 levels)



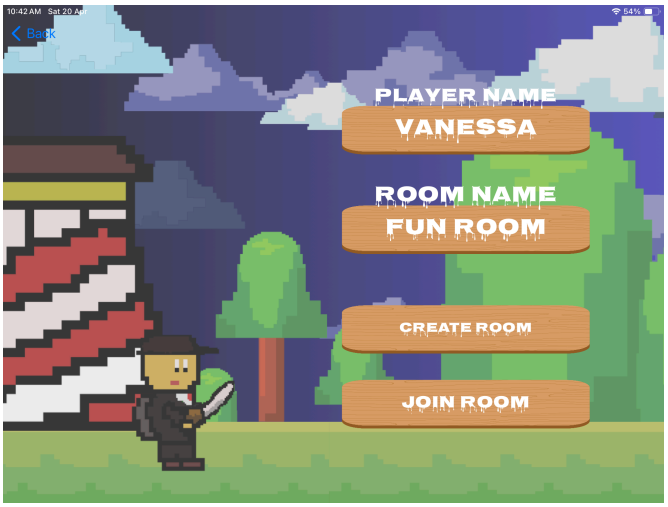
Gameplay View



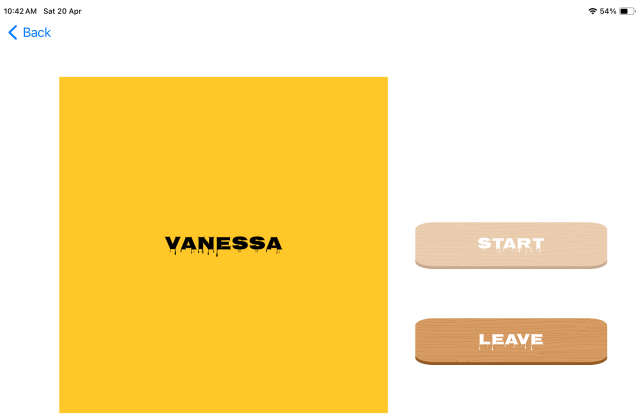
Multiplayer

Players can choose to play with their friends using the multiplayer mode. They will have the option to choose between creating a room or joining an existing room. They can join another room by knowing the name of the room that the other player gives. Each room can only have a maximum of 2 players. They can then start the game to play with other real players. While the default single player will be against the AI system.

Players need to create a name for the room and either press "Create Room" or "Join Room" if the room has existed



Players can press "start" when there are two players currently in the room to start the game for both players simultaneously

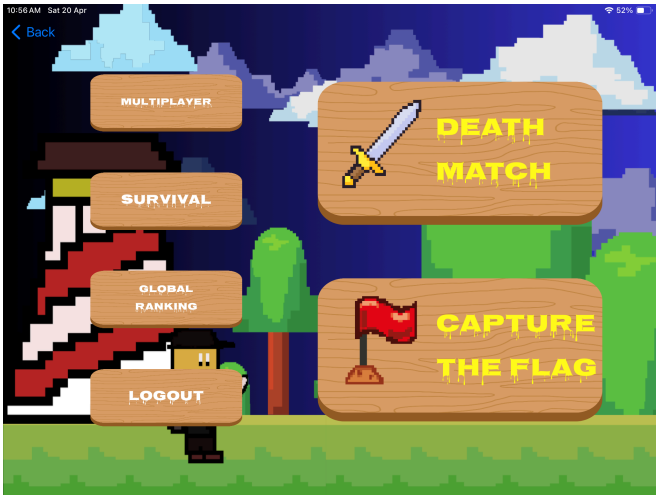


User Registration and Authentication

To ensure a personalized experience, players can register unique usernames and passwords to access the game's features. This registration process also involves email verification to guarantee the security and authenticity of each player's account. Once registered, players can log in securely to participate in the leaderboard / game rankings with other players in the community.

Leaderboard

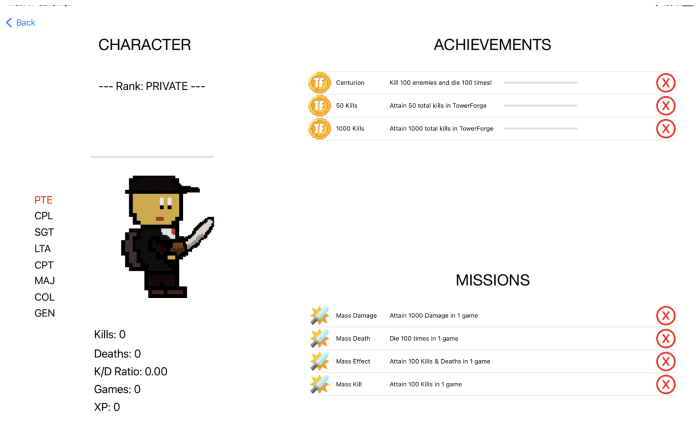
The leaderboard feature adds an extra layer of competition and motivation for players. It displays the top-ranked players based on various metrics such as overall score. Players can track their ranking and strive to climb the leaderboard by improving their skills. Additionally, the leaderboard fosters a sense of community by allowing players to view profiles of top-ranked players.

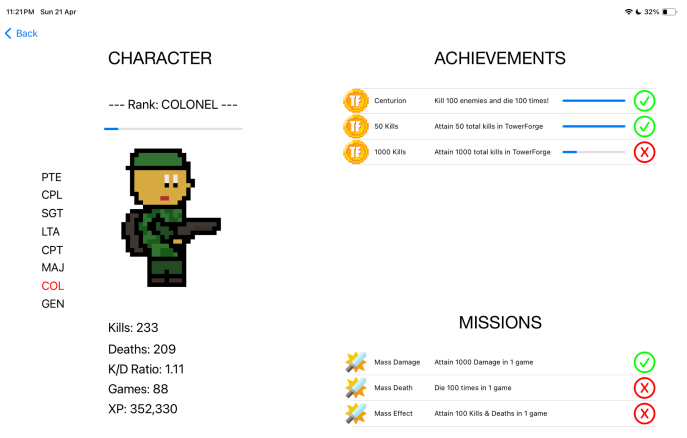




Player Screen & Progress Saving

The Player screen adds another layer of excitement to the game. The user can view their accumulated statistics, achievements and missions progress, and their own personal army-styled ranks. Ranks are shown based on the total XP gathered, and each action in game, like kills, games played, and damage dealt will all contribute the rank attained. As players progress further, their character will upgrade together with them - reaching an Officer Rank will don their character in military attire!



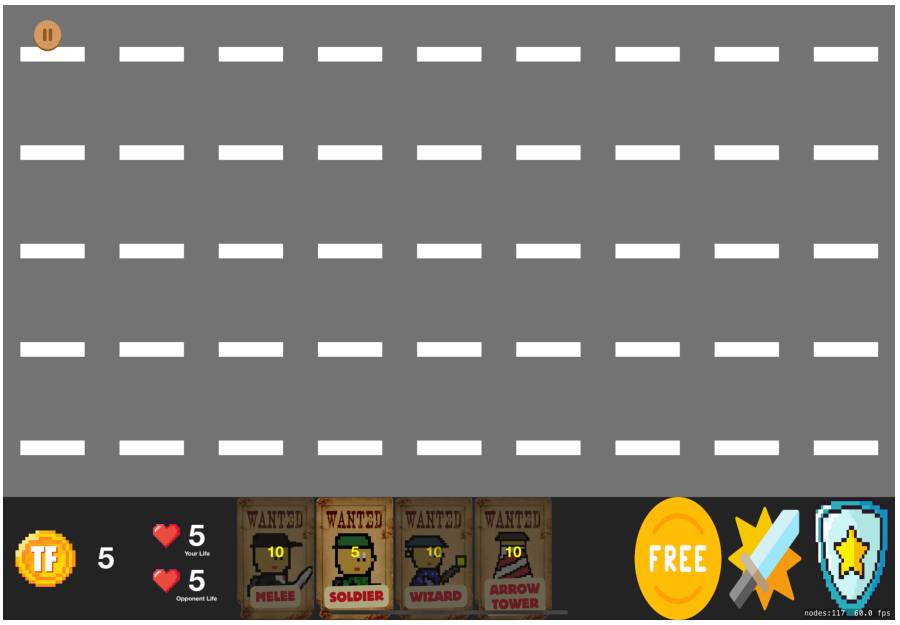


Demo

Shown below is the starting screen which will be shown on app launch.

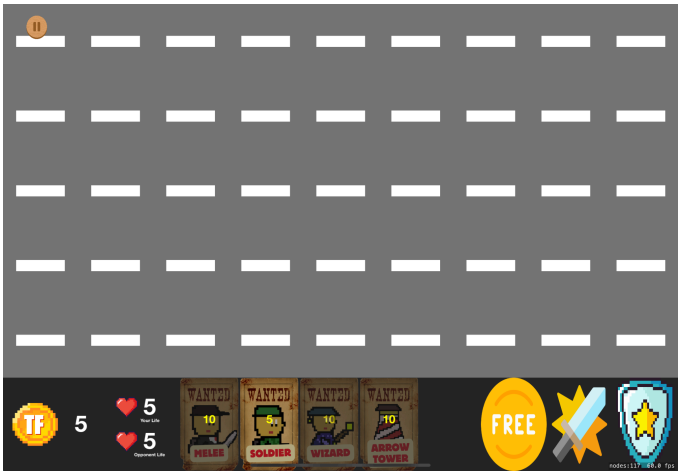


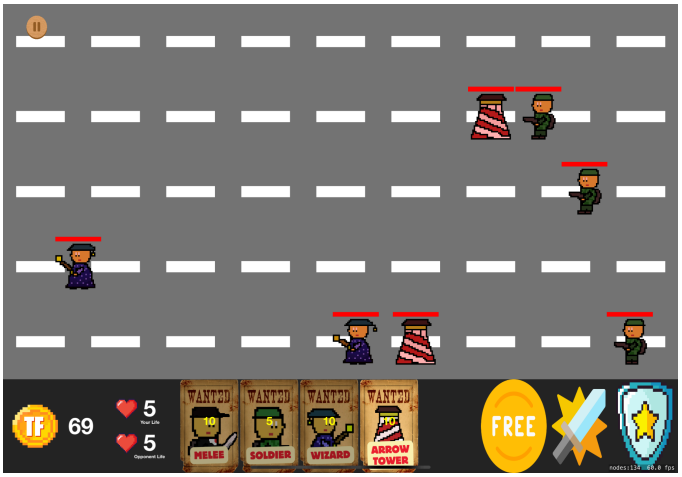
Clicking the play game button will bring you to the game scene.



As time goes by, the number of coins you have will increase. You can use these coins to purchase units. Additionally, as time goes by, enemy units will spawn on the right side of the screen and move towards the left side of the screen.

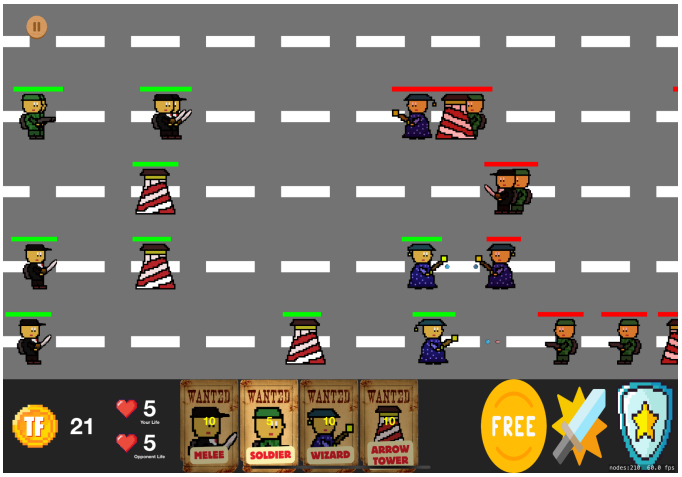
==





The left side of the screen indicates the user's "base" and the right side of the screen represents the enemy's "base". Enemy units will move towards the user's base and vice versa.

With coins, the user can generate more units and place them on the grid.



As shown on the right, as units come within firing range of each other, they will fire at each other, causing their health points to go down according to the damage dealt. After a certain number of hits, the units will "die" which will be represented by their disappearing from the game board.

Units that survive will continue to march towards their respective opponent bases.

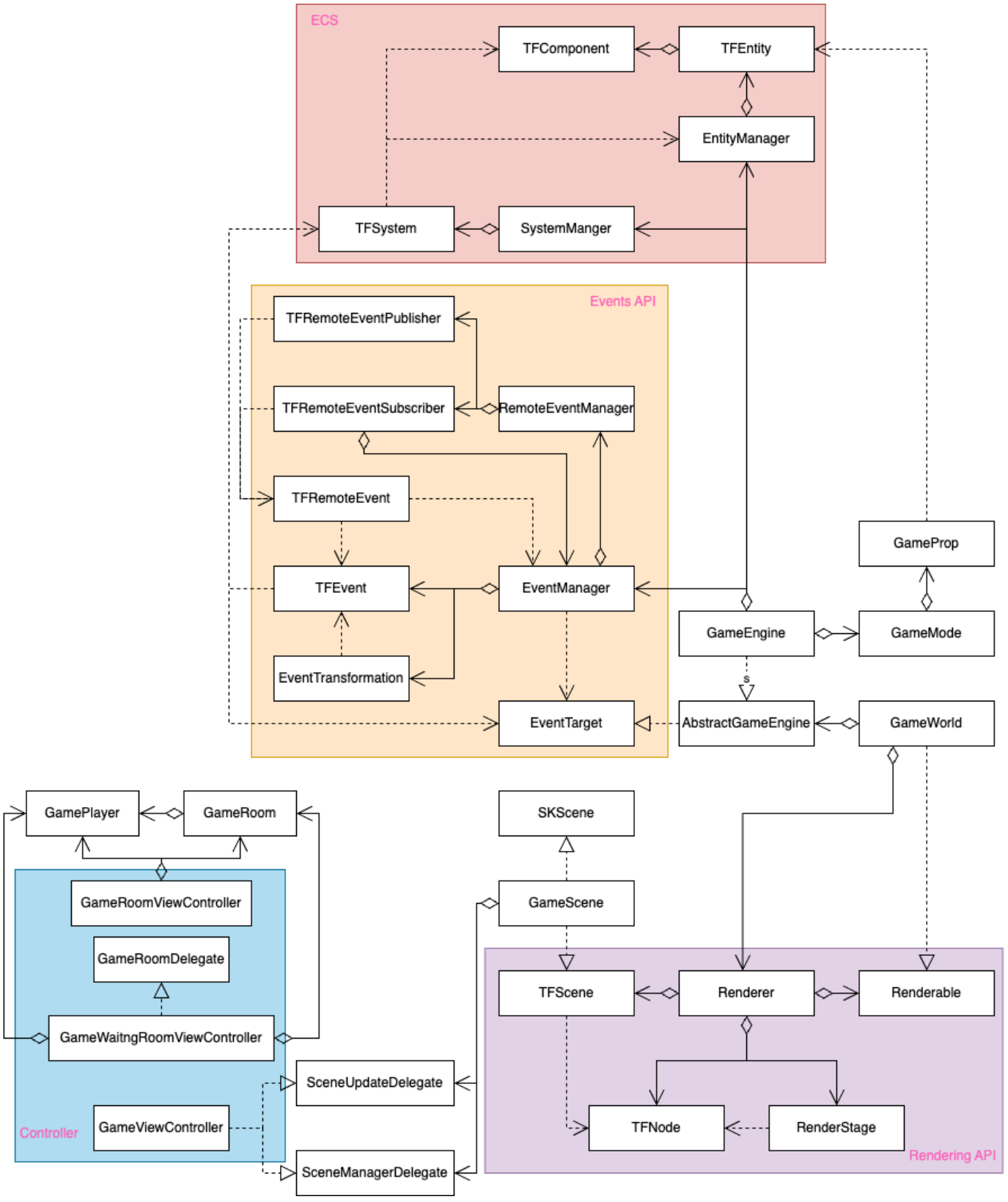
When a unit passes the left edge of the screen, the user's life points will decrease. This is denoted by the number beside the heart in the selection bar at the bottom. When this life point reaches 0, the user will have lost the game.

The same applies to the enemy. Thus, the goal of the game is to make the opponent's life points reach 0. The first one to make the opponent's life points reach 0 wins the game.*

4 – TowerForge Design Overview

Top Level Design

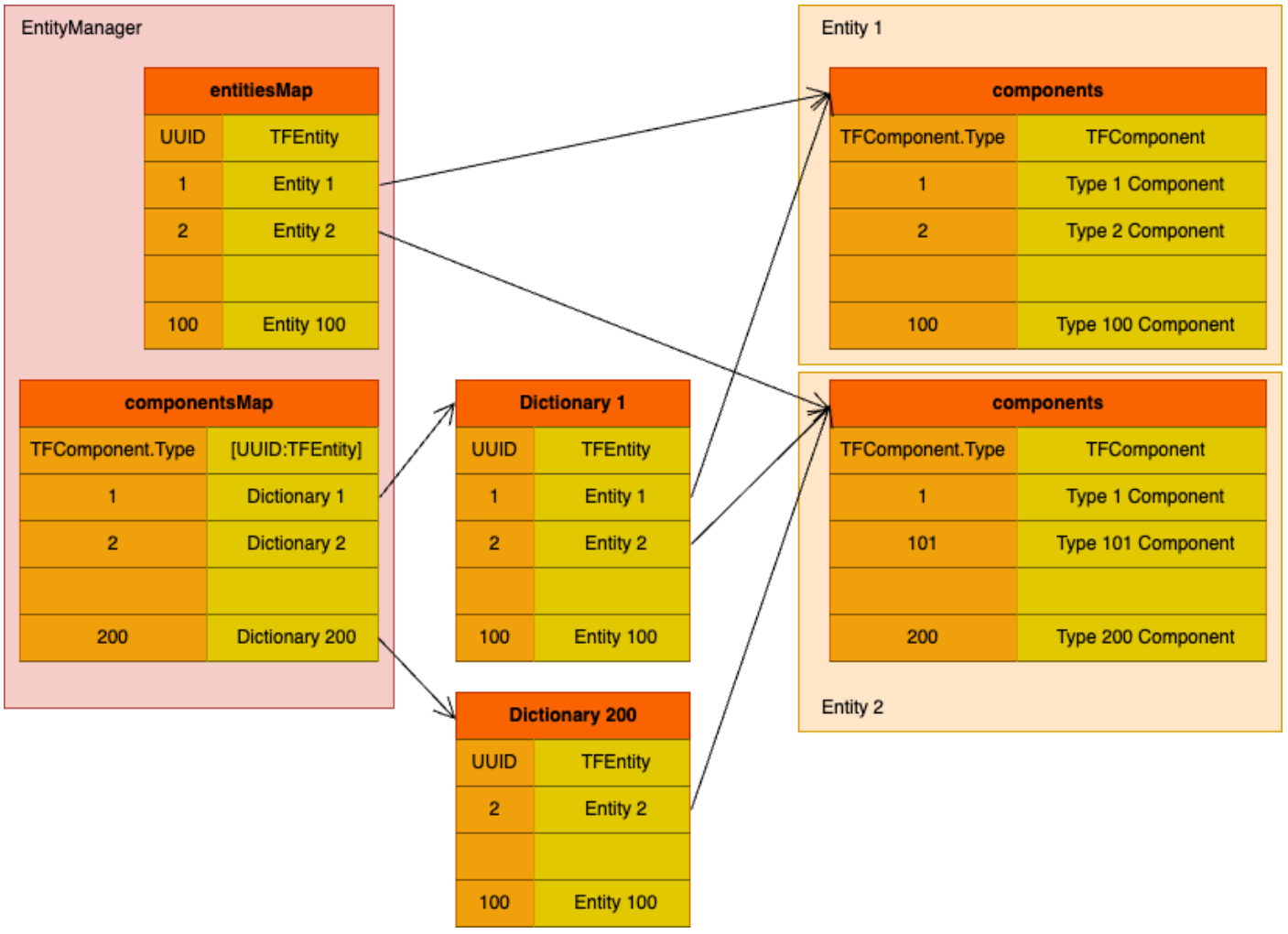
TowerForge consists of 3 main APIs. These include the Entity Component System(ECS), the Events API and the Rendering API. They are shown in Figure 1 below.



Entity Component System

TowerForge is developed upon a robust and well-known game architectural foundation that leverages the **Entity-Component-System (ECS)** paradigm to manage the game entities, behaviors and interactions. **TFEntity** represents the game entities and the fundamental building blocks of the game world / level. **TFComponent** encapsulates the data and behaviors while **TFSystem** orchestrates the processing of entities with specific component compositions so they are more like the features of the game object. This **ECS** framework promotes code reusability and enables simpler addition and removal of features and content.

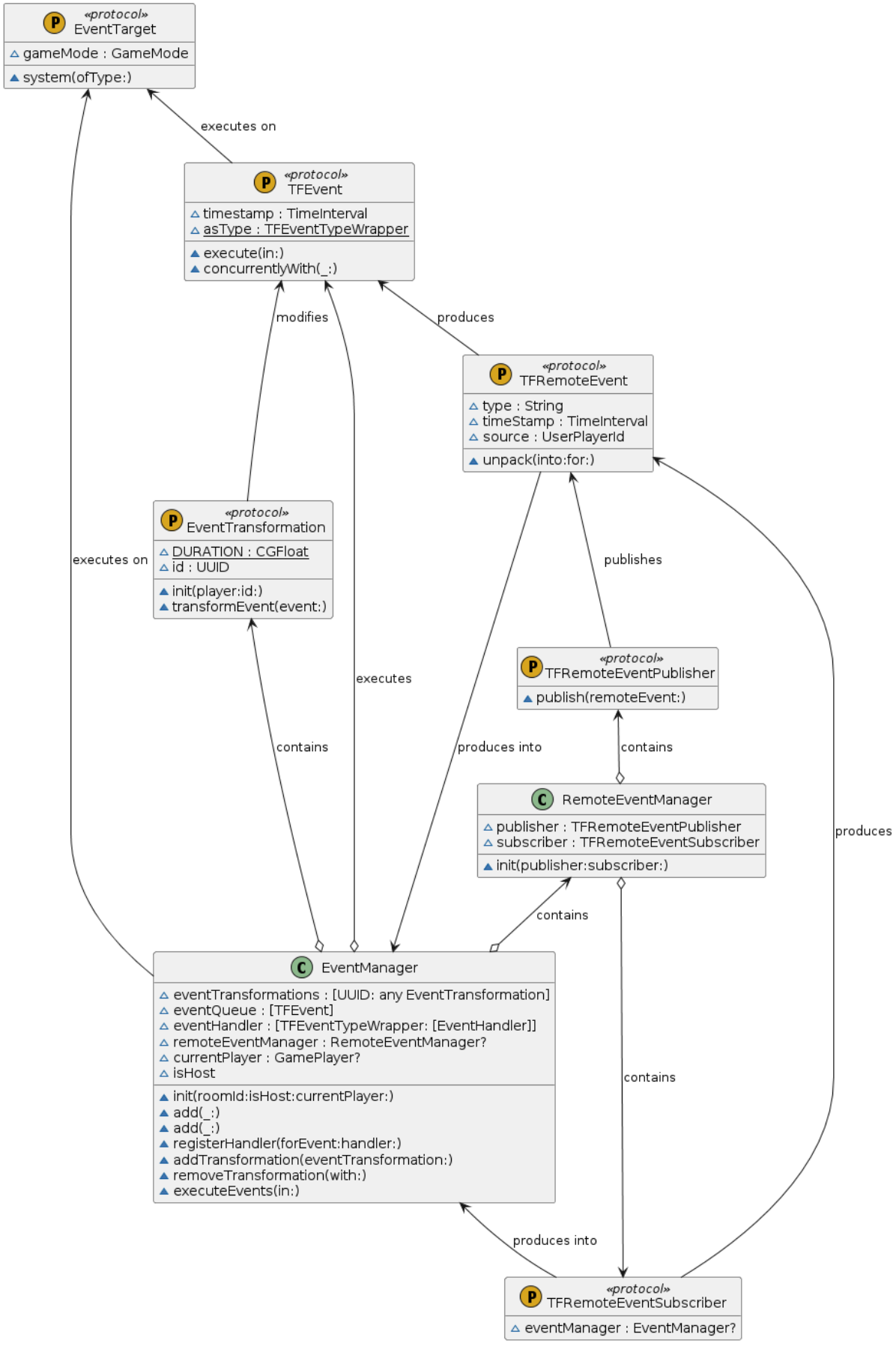
The **ECS** framework is supported by the **EntityManager** (Figure 2) which holds all the entities that exist in the GameWorld at any point in time. Any **TFEntity** must be added to the **EntityManager** for it to be able to interact with other entities and for it to be able to be rendered. We can then access and remove the entity using its unique identifier. Each entity will also contain a list of **TFComponent**s which defines the behavior of the **TFEntity** that it is attached to. All **TFComponent** have a weak association with their owner **TFEntity** so that we can easily query the **EntityManager** to get the entity that has that specific component.

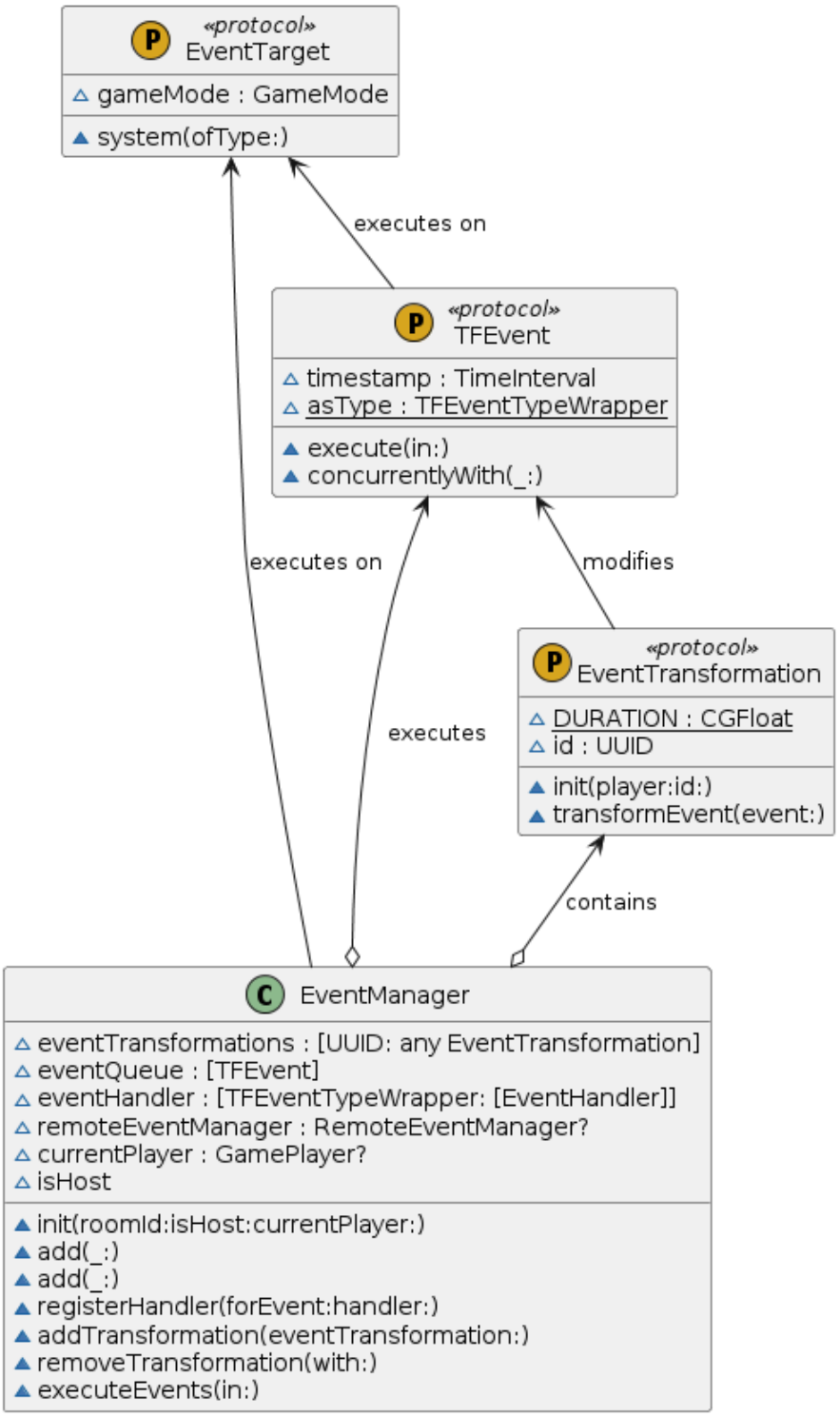


Each **TFEntity** and **TFComponent** are only to be modified by a **TFSystem**. This is to maintain the integrity of the data store in **TFEntity** and **TFComponent** which represents the game entity in the game world.

Events API

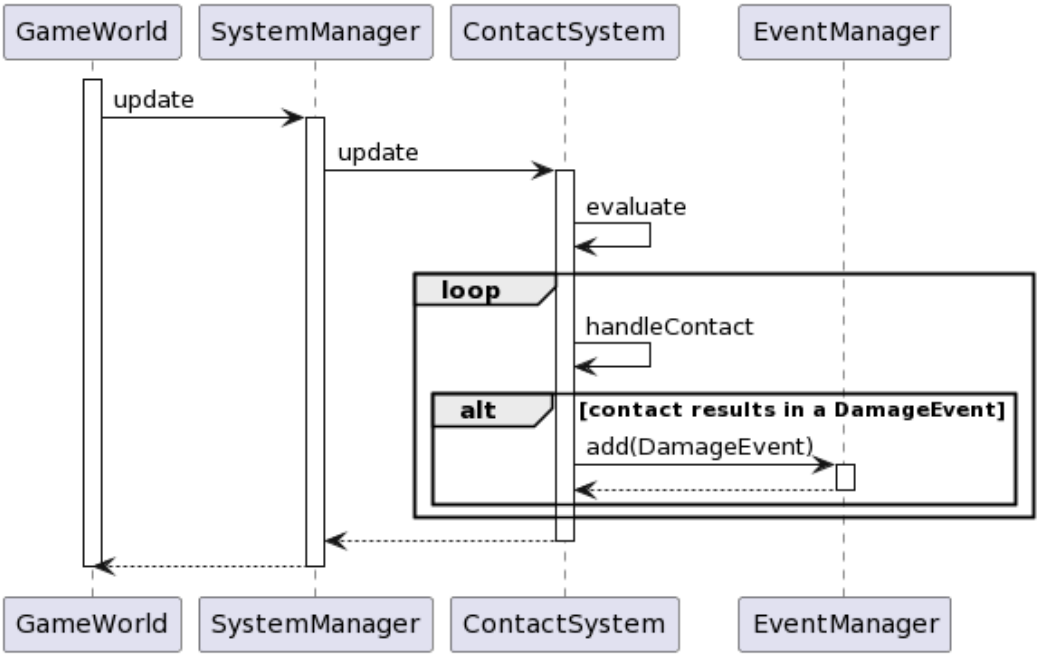
Events API serves as the means of communication between the different **TFSystem**s. The class diagram of the **Events API** is shown below.





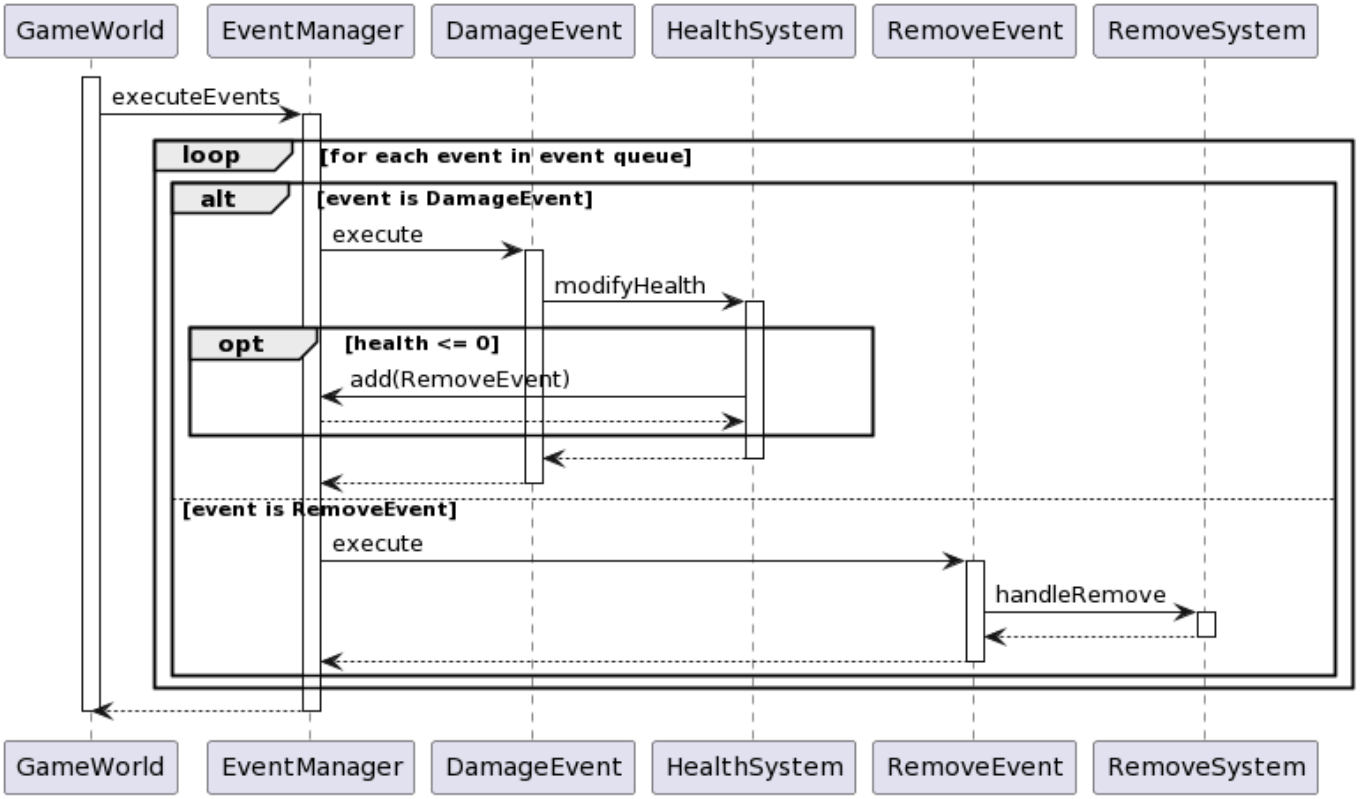
Each action that occurs in the **GameWorld** is represented by a **TFEvent**. These events, once created, are placed into the **EventManager** where they are added into an **EventQueue** which executes them in FIFO (First In First Out) order. Once called to execute, these **TFEvent**s will call the appropriate **TFSystem**(s), via the **EventTarget** protocol, which contain methods for the **TFEvent** to execute the action that they represent.

Below is a run through of the generation and resolution of a **DamageEvent** assuming there are no **EventTransformation**s registered.



At every frame update called by the **GameScene**, the **GameWorld** calls for updates by all the **TFSystem**s that are attached to the **GameWorld.**

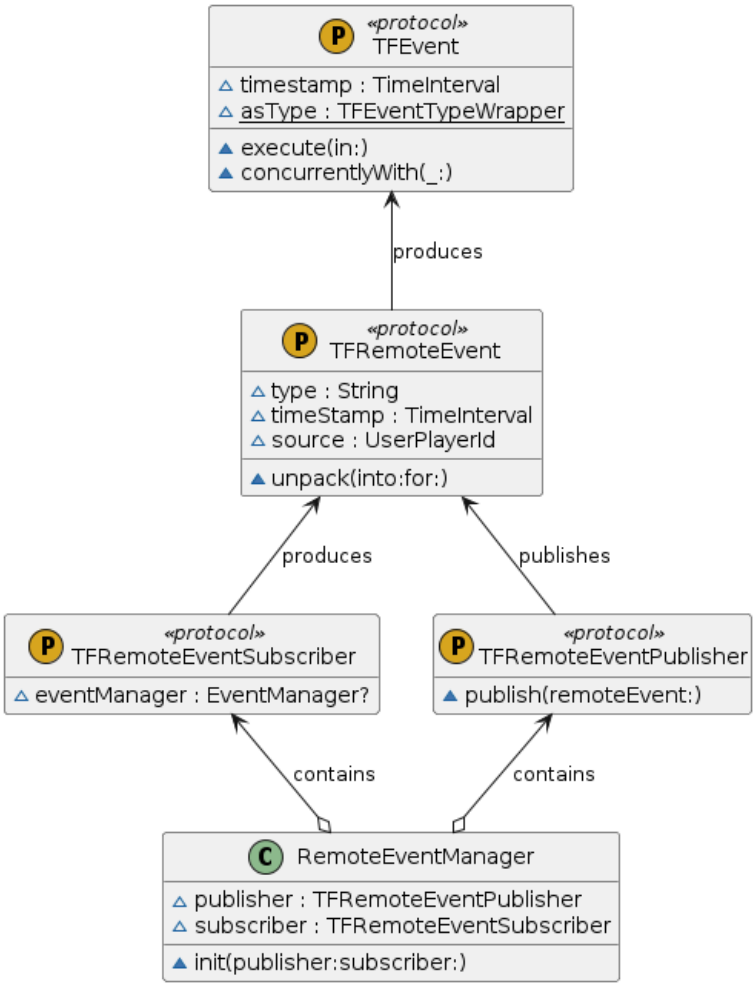
The **ContactSystem** iterates through all **TFEntities** that contain a **ContactComponent** and checks if there are any entities in contact. Using this, the **ContactSystem** invalidates any contacts for entities that are no longer in contact and evaluates contacts that are still occuring. This may result in the production of **TFEvent**s which is then added to the event queue in the **EventManager** to await execution.



After all **TFSystem**s have finished their updates, the EventManager then executes all the **TFEvent**s that have accumulated in the event queue. If the **TFEvent** is a **DamageEvent**, it calls the **HealthSystem** to update the health of the damaged **TFEntity** that contains the id specified in the **TFEvent**. Once the health is updated, the **HealthSystem** returns the resulting health of the **TFEntity**. If the resulting health is less than or equal to 0, a **RemoveEvent** for the damaged **TFEntity** is returned by the **DamageEvent** to **EventManager** which adds the **RemoveEvent** to the event queue to await execution.

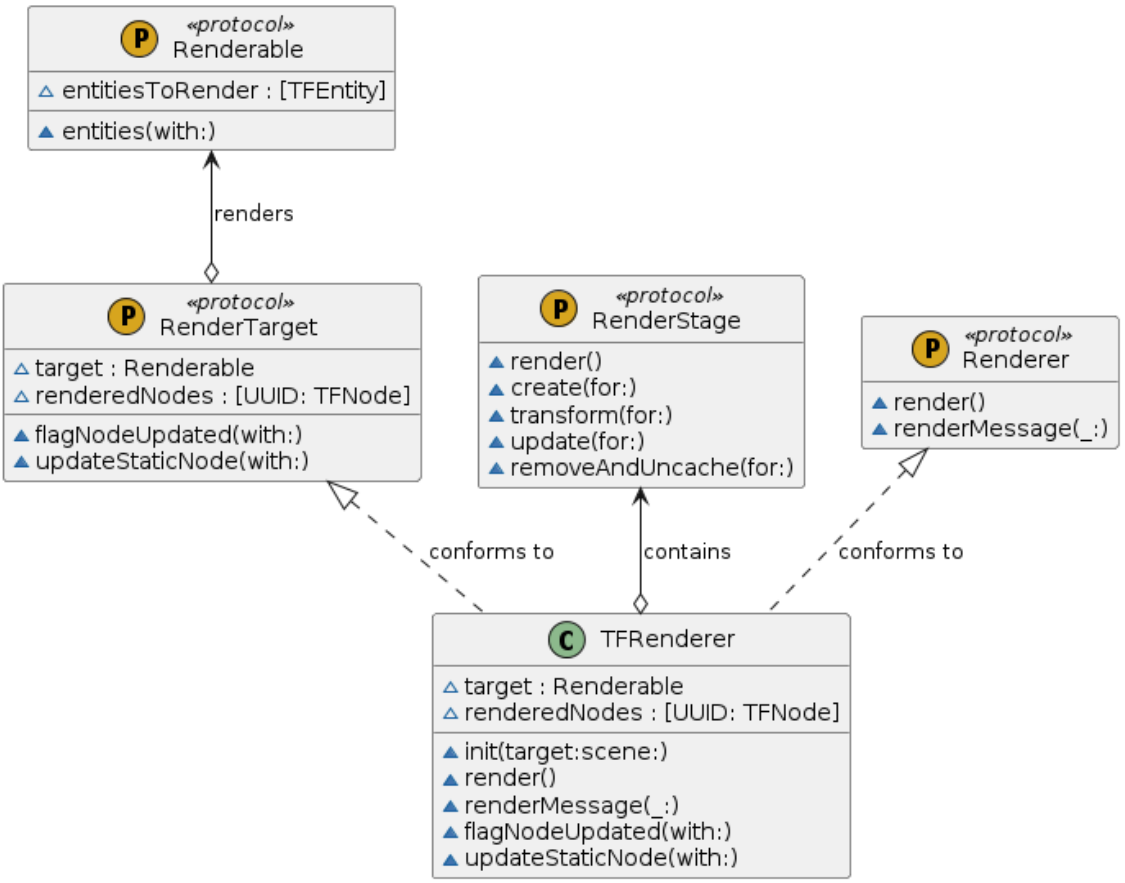
If the **TFEvent** is a **RemoveEvent**, it calls the **RemoveSystem** to remove the **TFEntity** with the id indicated in the **TFEvent** from the **GameWorld**. This would prevent other **TFEntities** from being able to interact with this **TFEntity** as well as being removed from the **Renderer** and will no longer be rendered.

The **EventsAPI** also consists of **EventTransformation**s which allows the **EventsManager** to execute **TFEvent**s with a pipeline. Before the **EventsManager** executes a **TFEvent**, it passes it to all the **EventTransformation**s that are currently registered which will modify the **TFEvent** if needed. This helps facilitate the implementation of the powerup mechanic in the game. For example, the **InvulnerabilityPowerUp** creates an EventTransformation that modifies all DamageEvents targeting entities that belong to the user player to become **DisabledEvents**. This is then registered into the **EventManager** and removed after the powerup has expired, preventing entities that belong to the user player from taking any damage while the powerup is active.



Moreover the **EventManager** also consists of the **RemoteEventManager** which is responsible for propagating and receiving **TFRemoteEvent**s produced by the **TFSystem**s. Once receiving the remote event, **TFRemoteEventSubscriber** will add the remote event which will unpack and produce a **TFEvent** to await execution in the next event cycle.

Rendering API



The **Rendering API** is responsible for bridging the connection between the **TFEntities** that exist in the **GameWorld** and the **GameScene** which displays the **GameWorld** to the users. The **Rendering API** currently consists of just the **Renderer** and **Renderable** which is a protocol which exposes a method for **Renderer** to retrieve the **TFEntities** that they need to render.

The **Renderer** caches the **TFEntities** that have already been rendered and only updates them as needed. This reduces the cost of updating the view as compared to removing all existing view elements and rerendering all of them. The criteria for the action that the **Renderer** executes for each **TFEntity** is listed in Figure 6 below.

TFEntity	**TFNode**	**Description**	**Action**
Exists	Not cached	TFEntity not rendered	Add and Cache
Does Not Exist	Cached	TFEntity removed from world	Remove and Uncache
Exists	Cached	TFEntity rendered	Update

Since different **TFEntities** can different vastly from one another due to possibly having a very different set of **TFComponent**s, it is hard for the **Renderer** to be able to accommodate all the different behaviors and display them properly. Thus, in order to accommodate this complexity, the **Renderer** uses the **RenderStage**s that have been added to it to convert the **TFEntities** that exist in the **GameWorld** into **TFNode**s to be rendered.

These **RenderStage**s acting as a **Render Pipeline** were inspired by the **TFSystem**s that are the core part of the **ECS** model. Just like how the **TFSystems** define the behavior of a **TFEntity** in the **GameWorld** depending on its **TFComponent**s, the **RenderStage**s define how the **TFEntity** looks when it is rendered. Each **RenderStage** operates on a single **TFComponent** type and ignores a **TFEntity** if it does not contain the required **TFComponent**.

At each render cycle, the **TFRenderer** will create a new root **TFNode** for all new entities. Subsequently it calls each **RenderStage** to render and update the **TFNode**s that they are responsible for. Each of the **RenderStage**s will get all **TFEntities** that contain the **TFComponent** that they are responsible for rendering and render each entity following the criteria denoted in Figure 9. If a new **TFNode** is created in the **RenderStage**, it is added to the root **TFNode** that was created beforehand. Each **RenderStage** will keep its own cache of entities that it has rendered and update them as needed. Once all **RenderStage**s have finished updating, any entities that are not updated will be removed from the scene.

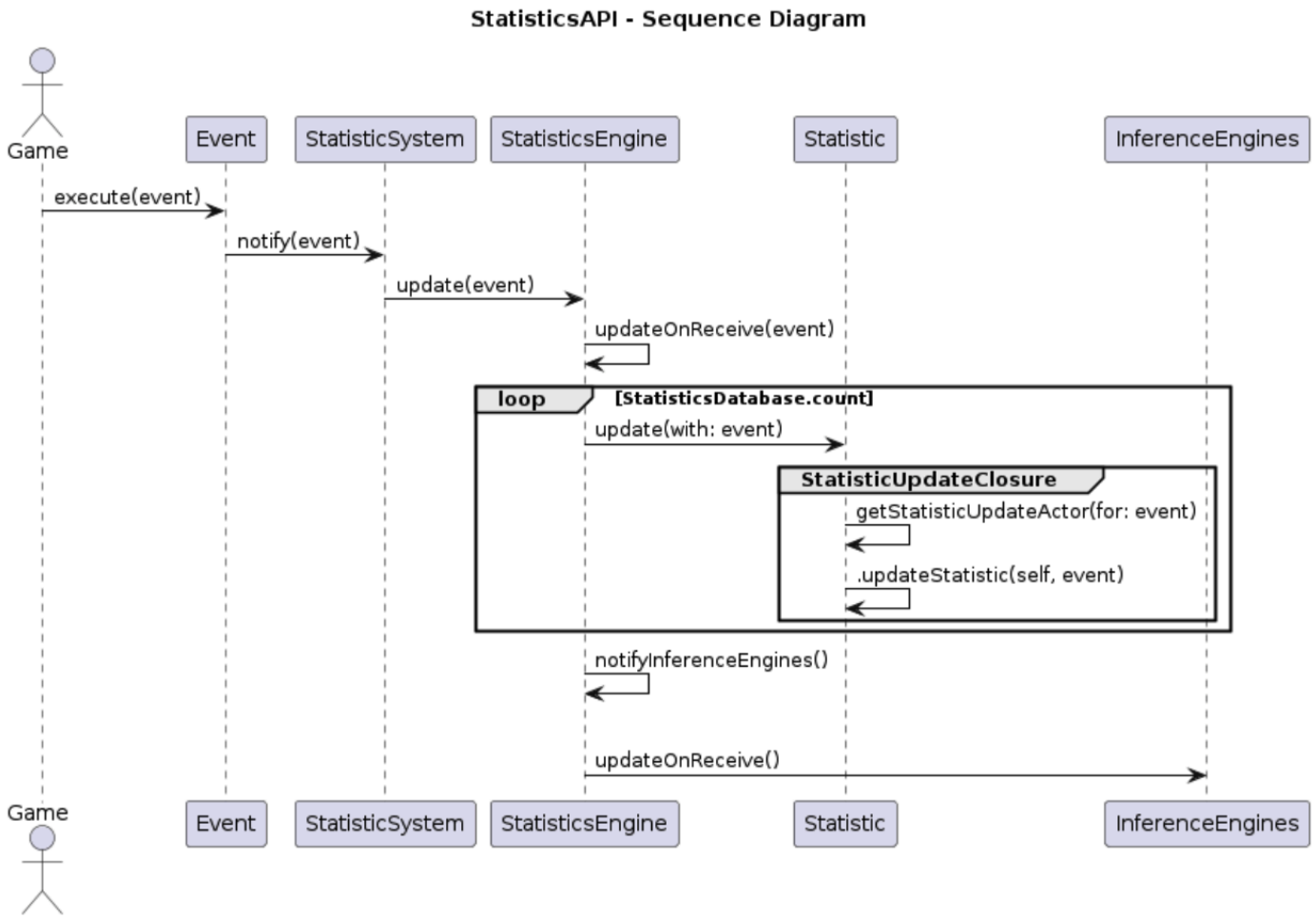
MetricsAPI

In TowerForge, the **MetricsAPI** is collectively responsible for collection, analysis and interpretation of meta-information about games and events that are executed in the course of a TowerForge game sequence. Given its wide-spanning nature, MetricsAPI is actually an abstract group of multiple different sub-APIs that closely interact with one another without being coupled together. Primarily, the MetricsAPI consists of two main Sub-APIs, which are the **StatisticsAPI** and the **InferenceAPI**.

StatisticsAPI

The **StatisticsAPI** is responsible for data collection and liaison with the **StorageAPI** (explained later). It is tasked with collecting information about game events, and interacts with the overall ECS Architecture via the **StatisticSystem** embedded within the ECS structure. As such, it follows the same event-driven architecture as the rest of the game.

As events are executed in the natural course of the game within the game's **EventManager**, specific relevant events will programmatically invoke the **StatisticSystem** in the course of their execution just like other events. The **StatisticSystem** will be instantiated similar to the other **TFSystem**s but with an additional variable that is an unowned reference to the game's **StatisticsEngine** which forms the core of the **StatisticsAPI**. The **StatisticsEngine** is then responsible for processing **Statistics** – determining the action to take based on the Event and whichever Statistics they might be associated with.



The above sequence diagram shows a brief overview of a single Statistic being updated. To summarize,

- The Event's execution will call the **StatisticSystem's** notify method, which will invoke **StatisticsEngine's** general **update** method with the executed Event.
- This will then called a private update method that retrieves an array of Statistics that correspond to that particular event type.
- Each Statistic within this array is passed the executed Event within its own update method.
- It will then retrieve a **StatisticUpdateActor** closure from its own **StatisticUpdateLinkDatabase**, which essentially contains information about the specific update action that corresponds to that given Event's type.
- This Actor is finally executed, which is what causes the final changes in the Statistic's value.
- The change is then propagated by the StatisticsEngine to the attached InferenceEngines, thereby updating their analysis based on the newly update **Statistics**

InferenceAPI

The **InferenceAPI** is responsible for analyzing the gathered information in the form of Statistics. Unlike the **StatisticsAPI**, the **InferenceAPI** does not have a concrete processing engine. Rather, the InferenceAPI is specified by the **InferenceEngine** protocol, which concrete processors can conform to. The **StatisticsEngine** contains a **collection of InferenceEngines** which will be notified upon changes to the current Statistics.

The **InferenceAPI** allows for varying and nuanced analysis of the same Statistic depending on the context of Analysis. For example, in the context of the **AchievementsEngine**, the **TotalKillsStatistic** can be used to determine completion of specific Achievements like "100 Kills" and "Centurion" Achievement (requires 100 kills and 100 death), while in the context of the **RankingEngine** a single kill would contribute a certain number of "XP" which can then be used to determine the current level of the player. Likewise, another custom **InferenceEngine** can be implemented to extract out specific information and present them in a different way.

Possible expansions include a "**GameObjectiveEngine"** that analyses the changes in Statistics for a given game and determines if certain in-game conditions are met, etc. Therefore, having decoupled the collection of Statistics from the inference thereof allows for easy expansion of TowerForge's gameplay in the future, even without having to predict what the parameters of the expanded gameplay are.

MetricsAPI and Single Responsibility Principle

Conventionally, developing games that require some form of Goal-driven gameplay like achievements or missions would store the information relevant to that particular goal within the goal itself. However, we decided that it is more architecturally sound to separate the two concerns, of collection of data and interpretation of data.

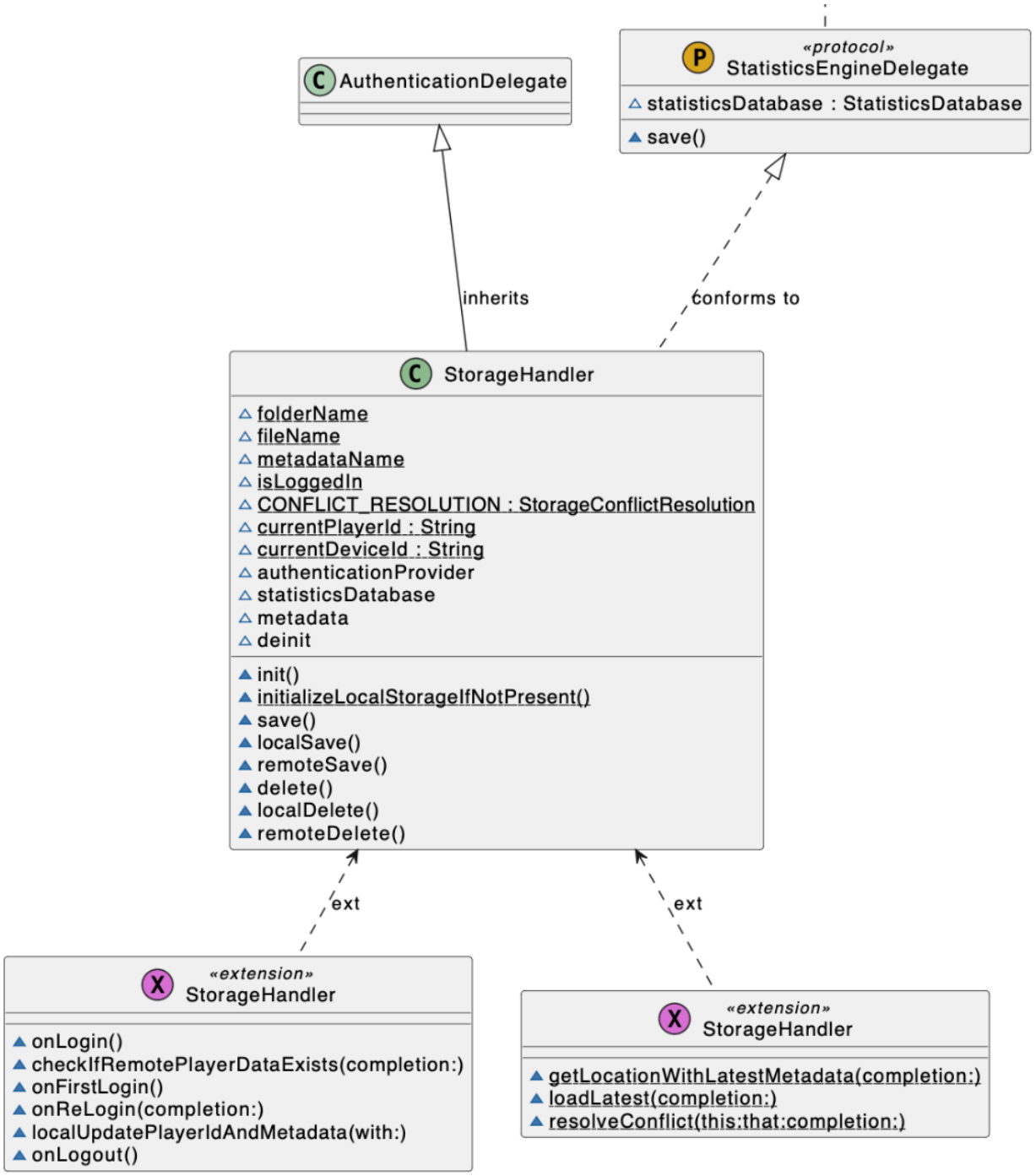
Not only is this approach more closely aligned with Software Engineering principles (i.e. Single Responsibility Principle), it also has rippling positive benefits, like being able to have tiered achievements that depend on the same base information, without having to duplicate the storage of that information. For example, the **TotalKillsStatistic** keeps track of all kills made by the player, and this can correspond to both the **FiftyKillsAchievement** and **HundredKillsAchievement**. Both Achievements will interpret the same data and determine their own completion accordingly.

StorageAPI

The **StorageAPI** is responsible for providing the data persistence functionality. Within TowerForge, there are 2 primary ways to store and retrieve information, via Local Storage and Remote Storage. In addition to the basic loading and storing of information, we have also implemented conflict resolution together with a robust dual-storage system. This serves as not just a **local cache** for data, but also as a **backup** and allows for a **double-redundant storage** in-case any one of the data sources fail.

StorageHandler

The **StorageHandler** forms the core of all data storage and retrieval. The StorageHandler handles all functions pertaining to local and remote storage, authentication, initialization, and conflict resolution. In order to better compartmentalize and organize code such that it conforms to SRP, StorageHandler is broken up into several extensions, and it is assisted by utility classes (RemoteStorage, LocalStorage, and their extensions) which abstract away some storage functionality.

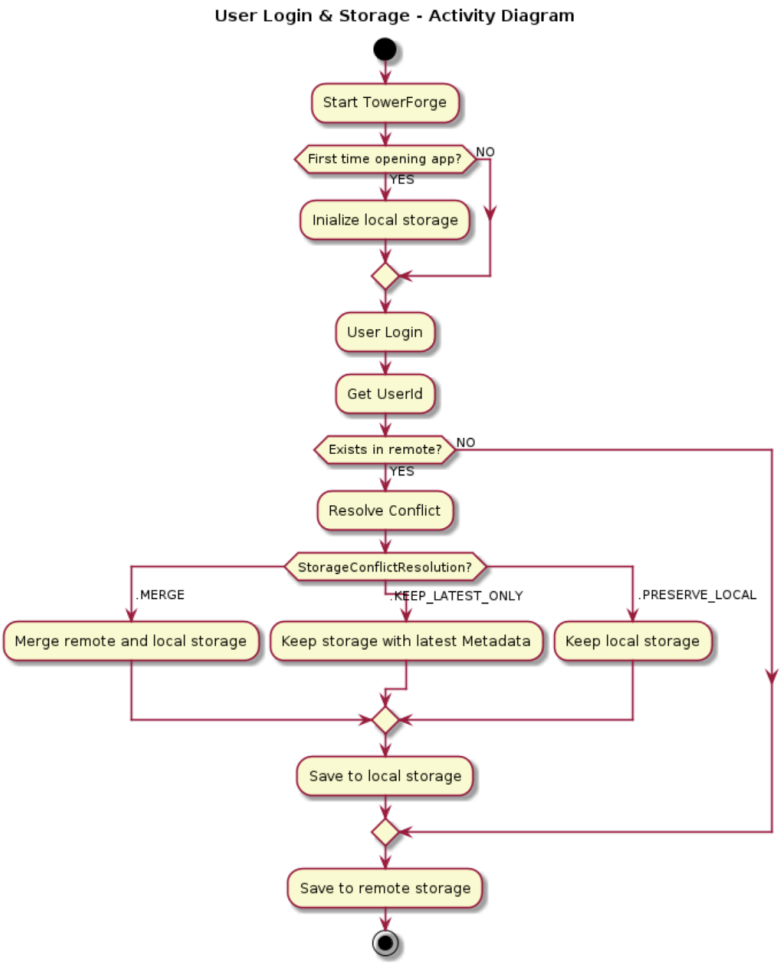


Metadata
In addition to the actual information stored, which is the **StatisticsDatabase**, a file called Metadata is also stored. This file contains information about the actual information itself, such as last update time, player id and device id.

Double-Redundant Storage & Local Cache
Local Storage is implemented with FileManager, and all storage operations write directly to file which is humanly readable via accessing the TowerForge folder locally. The moment the user opens the app for the first time, a local **Metadata** file and empty **StatisticsDatabase** are created.

Logging to the application will save the existing local storage information to a remote Firebase repository. From this point onwards, all information will be first stored to local storage (i.e. local cache) and then uploaded to Remote storage.

Player Progress Saving & Conflict Resolution
We wanted to ensure a consistent gaming experience for all users regardless of the device they are playing on, and thus we implemented a player progress saving system that takes into account the user's local gaming history and also their online history and created a system that can combine the two in a reasonable way.



In having a dual storage system, it is likely that data would become out of sync. For example, a player who has been playing locally will have some progress already saved. If they were to login to a new account, this progress would be carried over. However, if they were to login to an account that already has some

progress, it is not so straightforward, and some form of data conflict resolution has to be implement. In the above activity diagram, 3 different conflict resolution means are shown.

The first is **MERGE:** This option is merges two statistics database based on whichever statistic is higher. For example, database A with 10 wins and database B with 10 kills will result in database C - with 10 winds and 10 kills - being stored to file. The merge functions are directly implemented into each concrete statistic. This is the most logical way to combine the two databases as the player's overall gameplay experience will be the same

The second is **PRESERVE_LOCAL:** This option will preserve the local storage regardless of the values of the remote storage, and will override remote storage with the local storage. This is useful when Remote Storage is intended to be purely a back-up redundant storage and the user only intends to play on a single device.

The last is **KEEP_LATEST_ONLY:** This option reads both the local and remote metadata to consider which location has the most recently updated statistics and will override the other storage with the most recently updated.

Within the Constants class, the conflict resolution method can be changed programmatically, but it is set to MERGE by default.

Usage of Libraries

We utilized **SpriteKit**'s components such as **SKSpriteNode**, **SKTexture**, **SKLabelNode**, and **SKActions** to create adaptors for our game engine. For example, we developed **TFSpriteNode** and **TAnimatableNode** by building upon **SKSpriteNode** and **SKAction**, respectively. These adaptors provided additional functionality and customization options tailored to our game's specific requirements. With **TFSpriteNode**, we extended the capabilities of **SKSpriteNode** to suit our game's rendering needs, while **TAnimatableNode** allowed for more sophisticated animations and visual effects to render the game units.

Design Problems

Network API

Since we wanted our game to be more accessible, we opted not to use an authentication system, allowing any player to be able to access our game with just a nickname. However, because of this decision it is hard to determine the identity of the player from anywhere in the app. Due to this, we have to constantly pass the player id all around the game. Moreover, it is also hard to ensure the one publishing the **TFRemoteEvent** to be one of the players in the **GameRoom**.

Event Transformations

Our implementation of power ups utilizes the concept of **EventTransformation**s to iteratively transform events in the **EventQueue**. Due to the architecture of the system, all game events go through the event queue and thus the scope of power-ups that can be implemented is comprehensive enough. The downside of this approach, however, is that power ups are restricted to existing events in the queue and the effect is limited to other events. One way to work around this issue is to create custom events that might represent custom power-up effects in the game.

5 – Runtime Structure

Relevant Classes

GameWorld

GameWorld is where the **EntityManager**, **SystemManager**, **EventManager** is initialized and set up. In the initialization of GameWorld, we set up the various systems present in the game along with the systems manager which is used to process systems, the event manager which is used to process events, and the entity manager which is used to add the initial entities in the game. For instance, we set up the teams and player information (points, life) on game initialization. We also add the various systems using the initialized system manager.

EntityManager

We use an **EntityManager** to manage the entities in the game. The **EntityManager** contains an **entitiesMap** which uniquely identifies a **TFEntity** with a **UUID**. The systems can then call on the entity manager to perform operations (retrieval, addition, deletion) on the components in the ECS module. It also contains a componentsMap which maps **TFEntities** to a type of **TFComponent**, allowing for easy access to all **TFEntities** that contain a type of **TFComponent**.

SystemManager

The SystemManager is used to manage the various systems present in the game. The SystemsManager is responsible for adding or deleting systems, which is done on game initialization. Then, its primary function during the game run is to update the various systems. The various systems are responsible for their own updates. For instance, the MovementSystem will handle the movement of the movable components in the different entities present in the game.

Implementation of CheckRepresentation

Overview

EntityManager contains a dictionary where the key is the **UUID** of the element stored. **EntityManager** contains a dictionary called “entitiesMap” whose key is of the type **UUID** and the corresponding value is a **TFEntity**. Since each **TFEntity** contains a **UUID** as its unique identifier, this allows for fast retrieval of a given **TFEntity**, with O(1) average time complexity. It is possible to query a **TFEntity** without knowing any information about it.

TFEntity contains a dictionary where the key is the **TFComponent Type** and the corresponding value is a **TFComponent** so the same principle can be generalized to **TFComponent** as well.

Runtime Vulnerability

The universality of **UUID** and its inherent uniqueness (without requiring a central authority to validate its uniqueness) allows for multiple modules or systems that encapsulate **TFEntity** to exchange information about a specific **TFEntity** without directly exchanging copies of a **TFEntity** itself, by simply passing the **UUID** associated with that entity. However, this opens up the possibility of **UUID**s being mutated and thereby introducing a vulnerability in the system. Even though the **UUID** of each **TFEntity** is a *let* constant, the keys can be mutated resulting in a situation where one the key doesn’t correspond to the id within the **TFEntity** that is the value corresponding to the key.

CheckRepresentation Function

To fix this vulnerability, a checkRepresentation function is introduced wherever there is a usage of a value whose id is the key of a dictionary data structure used elsewhere in the codebase.

```
private func checkRepresentation() -> Bool {
    for (key, _) in entitiesMap where key != entitiesMap[key]?.id {
            return false
    }
    return true
}
```

Usage

This function can then be called to assert wherever that particular data structure is used. In our case, this corresponds to wherever **TFEntity** is contained, which is **EntityManager**. Whenever this data structure is modified, the checkRepresentation function can be asserted. The private nature of the function ensures that the integrity of the data structure can be ascertained without ever having to expose that data structure to test modules.

Entity Manager

```
func add(_ entity: TFEntity) {
    entitiesMap[entity.id] = entity
    for (key, _) in entity.components {
        if componentsMap[key] == nil {
            componentsMap[key] = [:]
        }
        componentsMap[key]?[entity.id] = entity
    }
    /// assert(checkRepresentation())
}

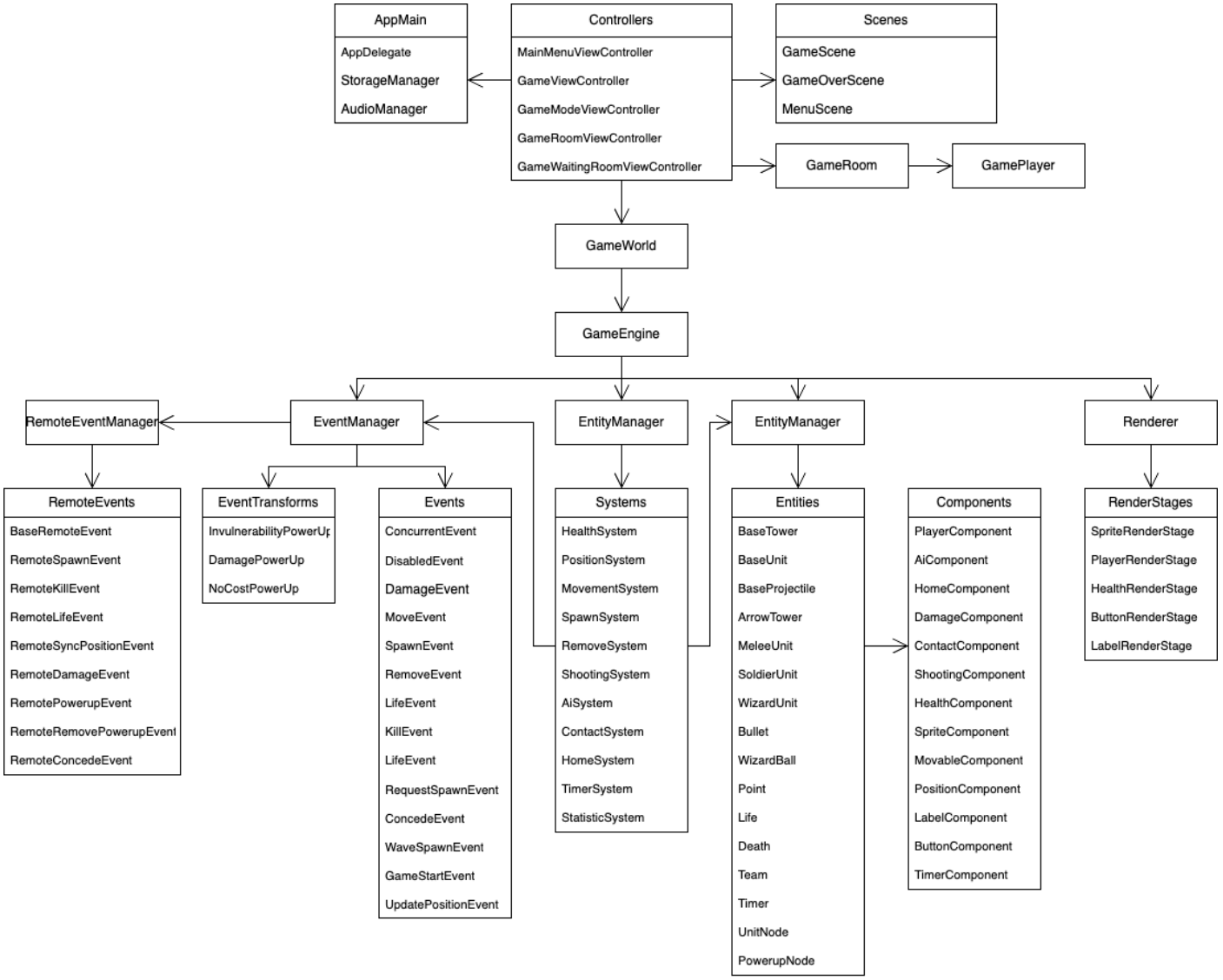
func removeEntity(with id: UUID) {
    guard let entity = entitiesMap.removeValue(forKey: id) else {
        return
    }

    for (key, _) in entity.components {
        componentsMap[key]?.removeValue(forKey: entity.id)
    }
    /// assert(checkRepresentation())
}
```

6 – Module Structure

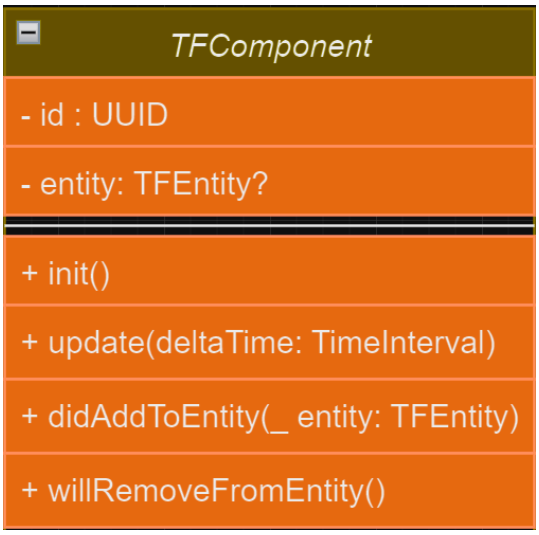
Module Structure and Diagrams

Below is the module diagram for the game:



ECS Framework

TowerForge follows the **ECS** framework by separating the fundamental design into three parts : Entities, Components and Systems. More about this, including the design patterns used are explained in the Design overview section.



TFComponent represents a characteristic or trait that the game entity has. Each component can override the update(deltaTime:) method to make updates to the entities and components which work well with the **Game Scene**'s update method.

Each component might belong to a **TFEntity**, hence it is a weak reference to the entity that this component belongs to. Components often need to interact with other components within the same entity through this way.

TFSystem is where the component system lies and is responsible for handling updates. Each **TFComponent** will have an associated **TFSystem**.

Unit Generator is used to spawn the units into the game. In this class, we have methods to spawn different units.

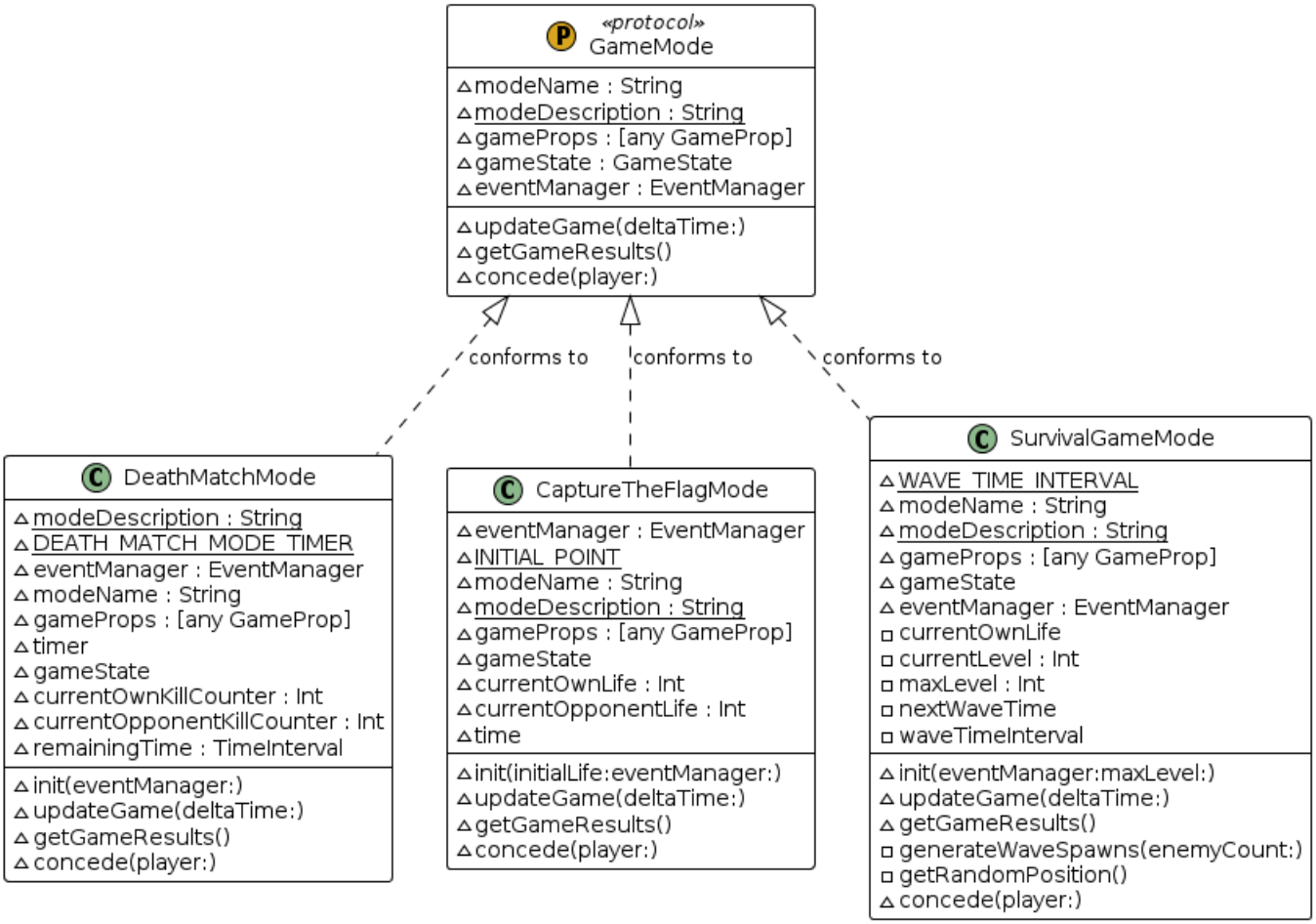
TFEvent represents an event that will be executed in the game. The game uses events to process changes in the game where each event corresponds to a single change in the game. For instance, the spawn event will spawn a unit in the game upon execution.

UnitSelectionNode is used to select the player's unit that will be deployed in the game. The player can choose from the units that are available in the game and this will be reflected accordingly on the screen.

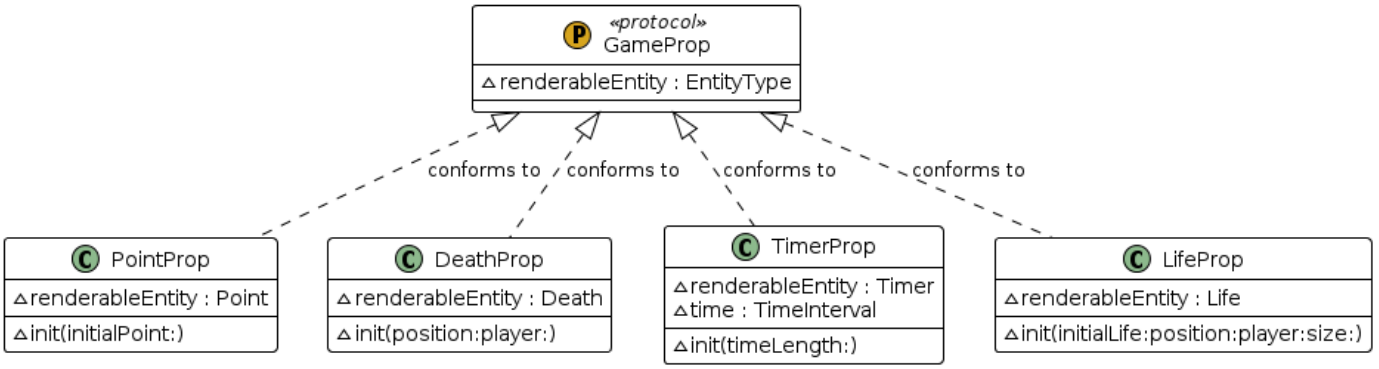
StorageManager is used as the adaptor for data persistence. The application access StorageManager via a singleton instance.

AchievementManager handles all achievement-related activities such as loading and updating Achievements. **Achievements** conform to the **Storable** protocol which allow them to be encoded and decoded with StorageManager.

Game Modes



The **GameMode** protocol defines the specific game play and objective of the game. The game modes that conform to this protocol like the **DeathMatchMode** class will then be able to offer unique challenges. The **GameModeFactory** implements the Factory pattern that is responsible for creating instances of different game modes based on the type. This then helps to promote modularity and extensibility hence allowing us to easily add new game modes in the new future. Each game might have different HUDs and features and hence the **GameProp** protocol is added to encapsulate the behavior of in-game HUD entities that serve as a static metric to the players.

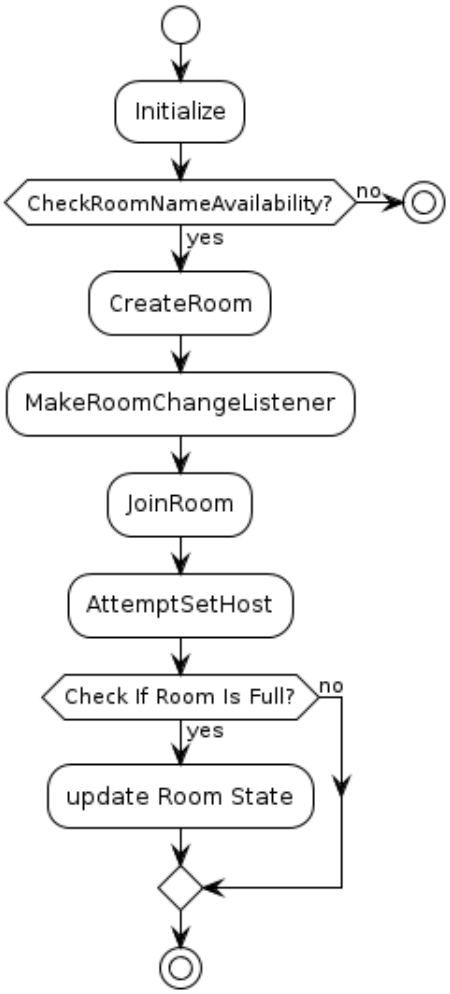


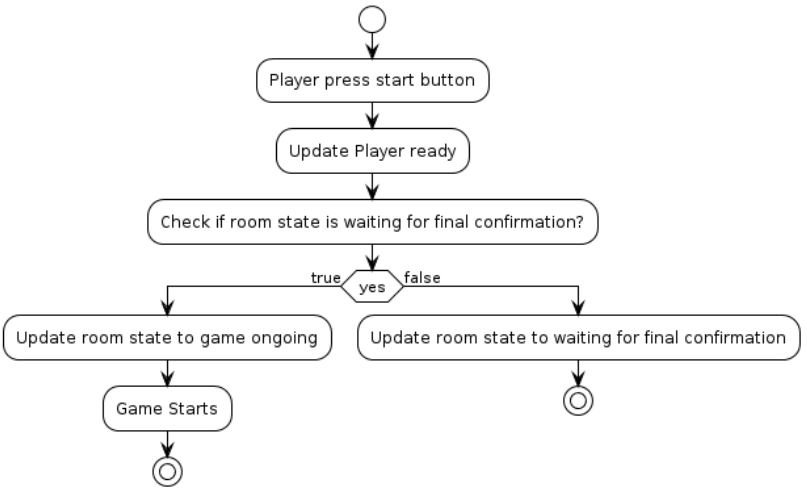
Hence, we can easily add new properties and customize the gameplay elements easily as needed. Furthermore, within the initialiser of each **GameMode**, event handlers are registered to the gameplay where certain callbacks are resulted if the specific event type happens in the game world.

```
eventManager.registerHandler(forEvent: KillEvent.self) { event in
            if let killEvent = event as? KillEvent {
                // Check if the event reduces life
                if killEvent.player == .ownPlayer {
                    self.currentOpponentKillCounter += 1
                } else if killEvent.player == .oppositePlayer {
                    self.currentOwnKillCounter += 1
                }
            }
        }
```

For instance, in the **DeathMatchMode**, when a **KillEvent** is triggered, the event handler updates the kill counter for each player. Here, the **EventManager** bridges the gap between event-driven interactions and logic within each game mode that might influence the progression and outcome of the game.

Game Room





The **GameRoom** class serves as a backbone to manage the players within the rooms. Before creating a room, the initializer will check if there is such an existing room in the database already in order to prevent duplicate rooms. If it is available, then data will be posted to the Firebase and a listener will be set up to listen for changes in the database. These changes are established through the makeRoomChangeListener method. The **GameRoomDelegate** protocol enables the other objects to receive notifications when the room changes so that the delegates can react to events and update the UI when the player joins or leaves the room.

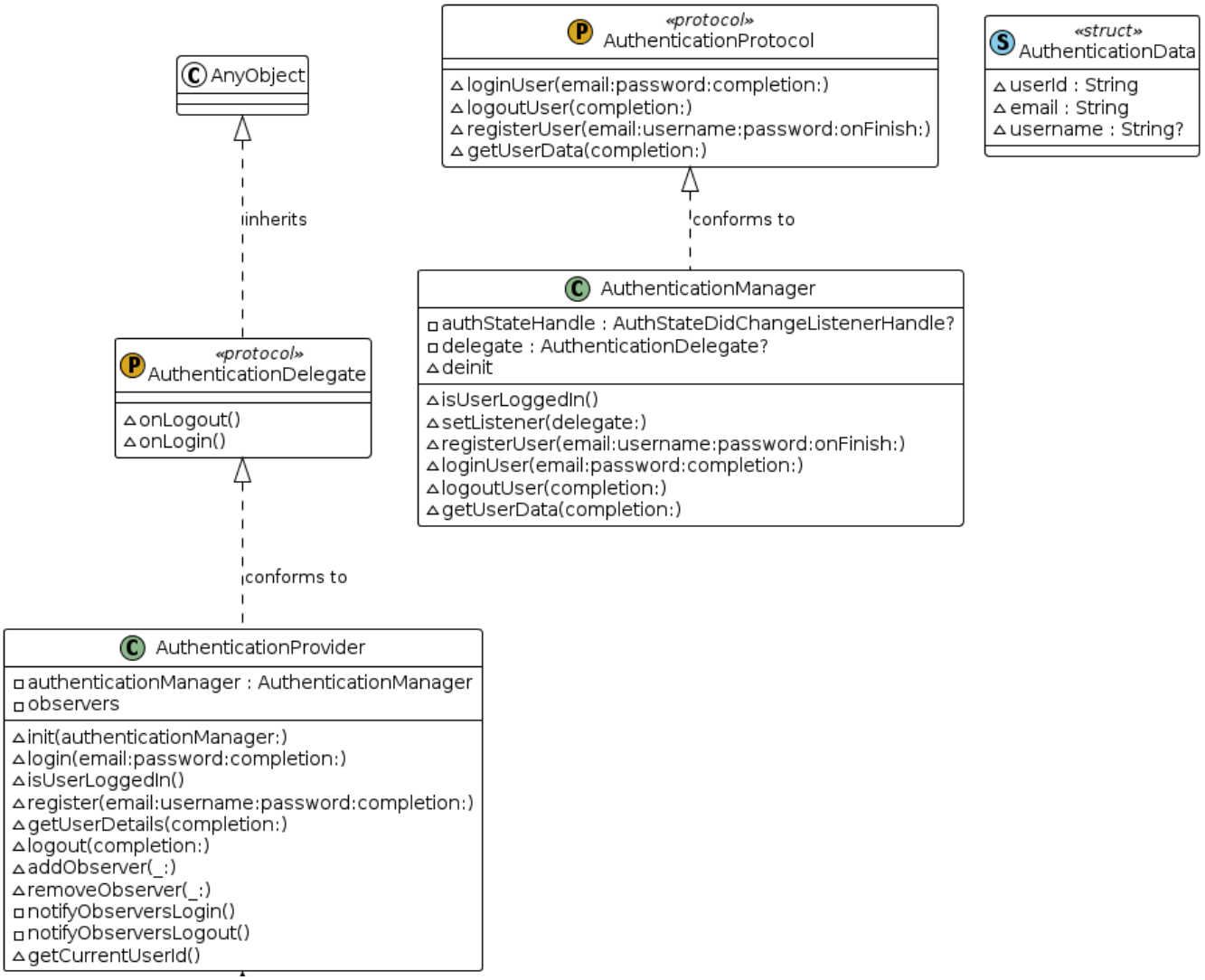
The player that creates the **GameRoom** will initially be selected as the host. That means that the state of their game will be regarded as the single point of truth. However, if there is another player in the room and the current host decides to leave the room, the other player will become the host instead.

Game Network

The game network serves as the form of communication between players playing in the same **GameRoom**. They do so via the use of **TFRemoteEvent**s. Each player will have their own **EventManager** that will contain a **RemoteEventManager** if they are playing in multiplayer. This **RemoteEventManager** will contain a **TFRemoteEventPublisher** and a **TFRemoteEventSubscriber**, allowing each player to upload and receive **TFRemoteEvents** from the other player. For example, if a player attempts to spawn an entity, it will create a **RemoteSpawnEvent** and publish it to the network server via the **TFRemoteEventPublisher**. This will then be propagated to all players that are subscribed to the same **GameRoom**, allowing them to receive the **SpawnEvent**, spawning the appropriate entity on their local copy of the game.

In each game, one of the players will be appointed as the host and will occasionally post sync updates to the network to be propagated to the other players to sync their game to. One of these syncs is the **RemoteSyncPositionsEvent** which will collect the positions of all **TFEntities** except for projectiles and upload them to the network. These positions are then received by the other player's device which will use the positions to synchronise their **TFEntities** against.

Authentication Manager

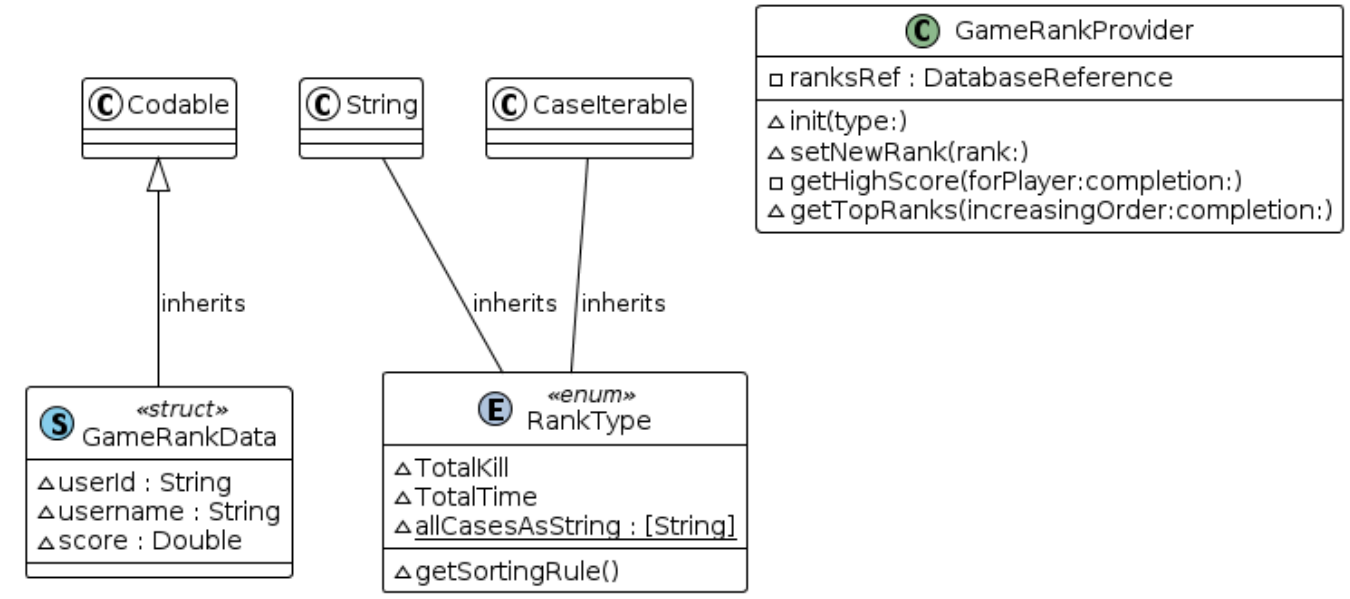


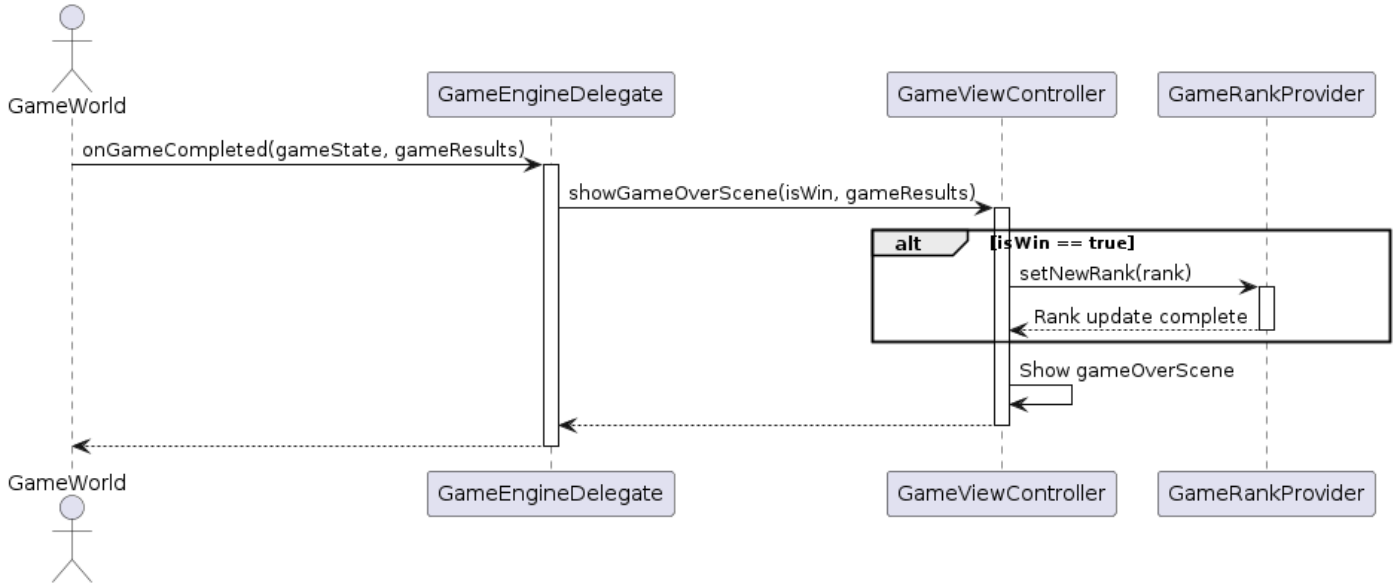
In order to participate in leaderboard rankings, users need to authenticate and register themselves into the system. Integrating with Firebase Authentication APIs, the **AuthenticationManager** encapsulates the authentication logic. While not explicitly implemented as singleton, the **AuthenticationManager** demonstrates a singleton-like behavior to ensure centralized authentication logic and state management so it acts as a single source of truth where all components interact with the same manager.

The **AuthenticationProvider** class serves as the subject or publisher that maintains a list of observers hence demonstrating an observer pattern like design. This class acts as a higher-level abstraction, offering an interface for interacting with the game. When there is an authentication event occurring, the **AuthenticationProvider** will notify the registered observers

To facilitate the communication between the manager and other components, we use the Delegate pattern so that observers can react to the authentication events when users login or logout. This acts as an interface for the observers to implement.

Leaderboard Provider + Game Ranking





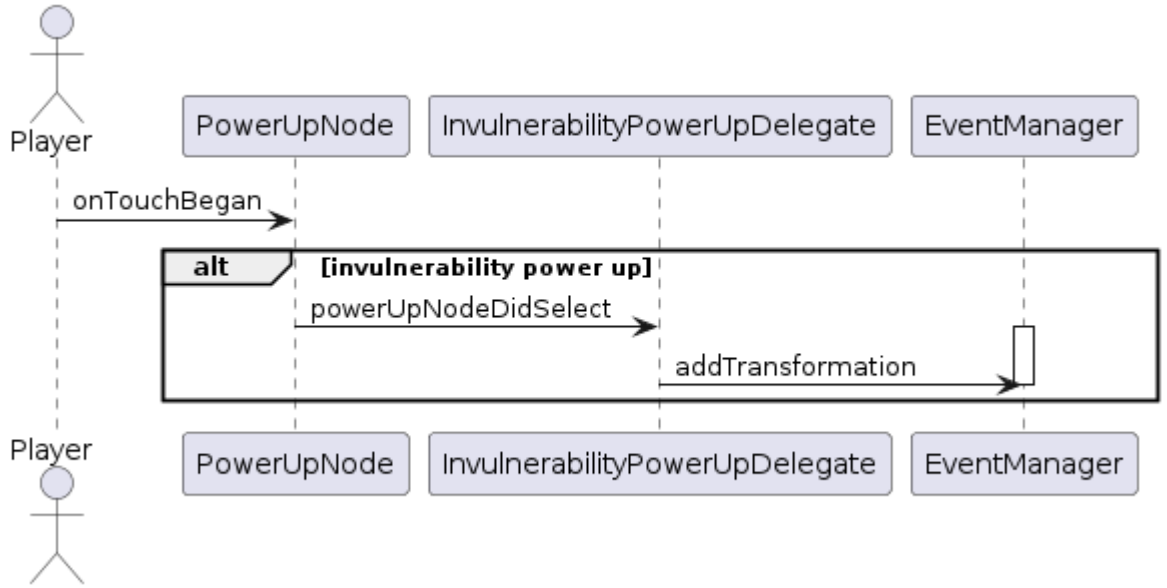
The **GameRankProvider** class serves as a repository for leaderboard-related data and operations, encapsulating the logic for interacting with the underlying data storage within a single abstraction. This pattern promotes separation of concerns by isolating data access logic from the rest of the application, enabling easier maintenance and testing. Additionally, the provider abstracts away the details of database operations, providing a clean and consistent interface for setting new ranks and retrieving top ranks without exposing the underlying implementation details. By adhering to the Repository pattern, the leaderboard provider achieves a modular and flexible architecture that enhances code maintainability, scalability, and testability. It acts as a bridge between the logic and the data which are responsible for querying and manipulating data from the source.

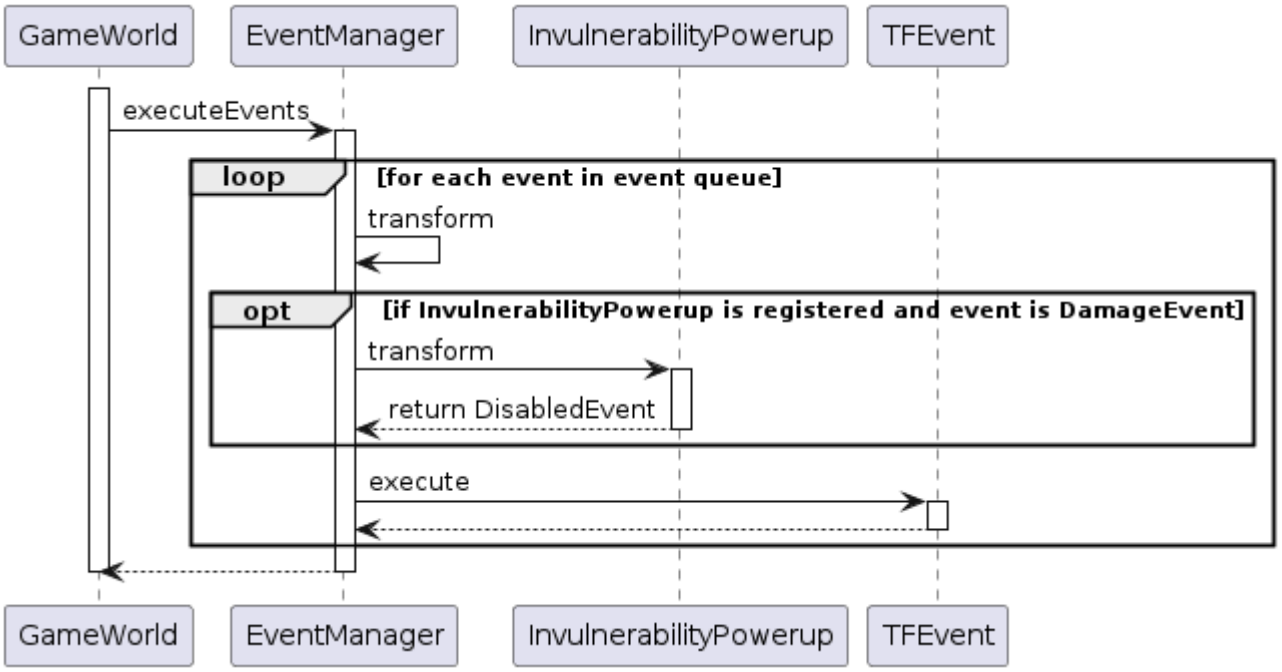
Power Ups

To facilitate the addition of power-ups in the game, we added an event pipeline under **EventManager** that events have to undergo before execution (refer to Figure 3 for the full class diagram). Each event in the event queue will undergo any **EventTransformation**s that are present in the list of event transformations under **EventManager**. Due to the clear separation of concerns between the Entity Component System and the Events API, all events in the game are handled by the **EventManager**. Hence, each power up will not have to gain access to the relevant systems and can instead be defined as **EventTransformation**s on the game events that it affects. In this way, the game behavior can be modified by power ups without having to interface with the entities themselves. Furthermore, each power up is singularly defined and conforms to the **EventTransformation** protocol, so we can utilize polymorphism to define power ups.

The implementation of power ups as **EventTransformation**s follows the chain of responsibility pattern. Each transformation either chooses to handle the existing event or pass it along to further transformations. This helps to reduce the complexity of handling multiple power ups that are active in the particular order that they were activated.

To activate power ups, we used the delegate pattern which will be called by the respective UI elements to activate the power ups. Each power up will define the duration for which it is active, and the delegate is also responsible for removing the power up after the duration has passed. We also utilized the factory pattern to initialize the power up delegates in the selection node.





Invulnerability Power Up

Each damage event in the game is handled by the execution of a **DamageEvent**. When a unit collides with another unit, a **DamageEvent** is created and appended to the event queue. When the invulnerability power up is active, all **DamageEvent**s on the player's units are disabled and transformed to a **DisabledEvent**. Hence, the player's units become invulnerable for a set period of time, defined in **InvulnerabilityPowerUp**.

Damage Power Up

When the damage power up is active, all **DamageEvent**s on the opponent's units are scaled up according to a predefined factor and transformed into a new **DamageEvent**.

No Cost Power Up

This power up enables the user to spawn units without any cost. Each spawn event requested by the user is handled by the execution of a **RequestSpawnEvent**. When a user has the no cost power up active and adds a **RequestSpawnEvent** to the queue, the power up transforms the **RequestSpawnEvent** to a **WaveSpawnEvent** which does not consume any coins available to the user. When the **WaveSpawnEvent** is executed, the unit spawns for the player without incurring any cost to the player.

Future Power Ups

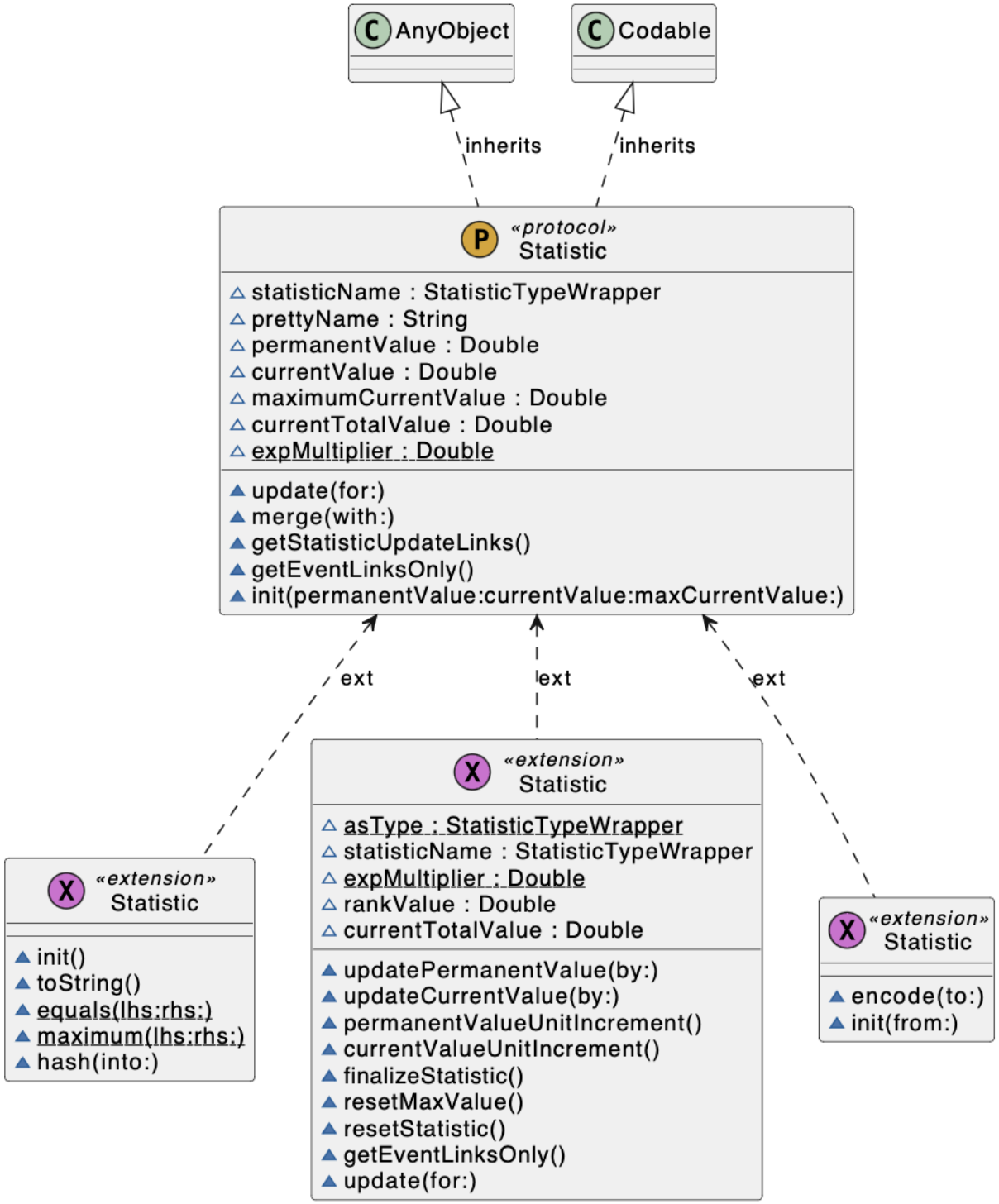
The benefit of the current implementation of power ups is the flexibility of the system. To add a power up, developers can define their own **EventTransformation** and operate on any event in the game as they wish. There is also the option to create custom events for custom power-ups that do not correspond to any existing events in the game. For example, power ups that affect movement can transform the **MovementEvent**s. The possibilities are endless due to the flexible design.

Statistics Data structures

As mentioned in the StatisticsAPI section, the Statistics Module consists of several key classes and Data structures used for Statistics.

Statistic:

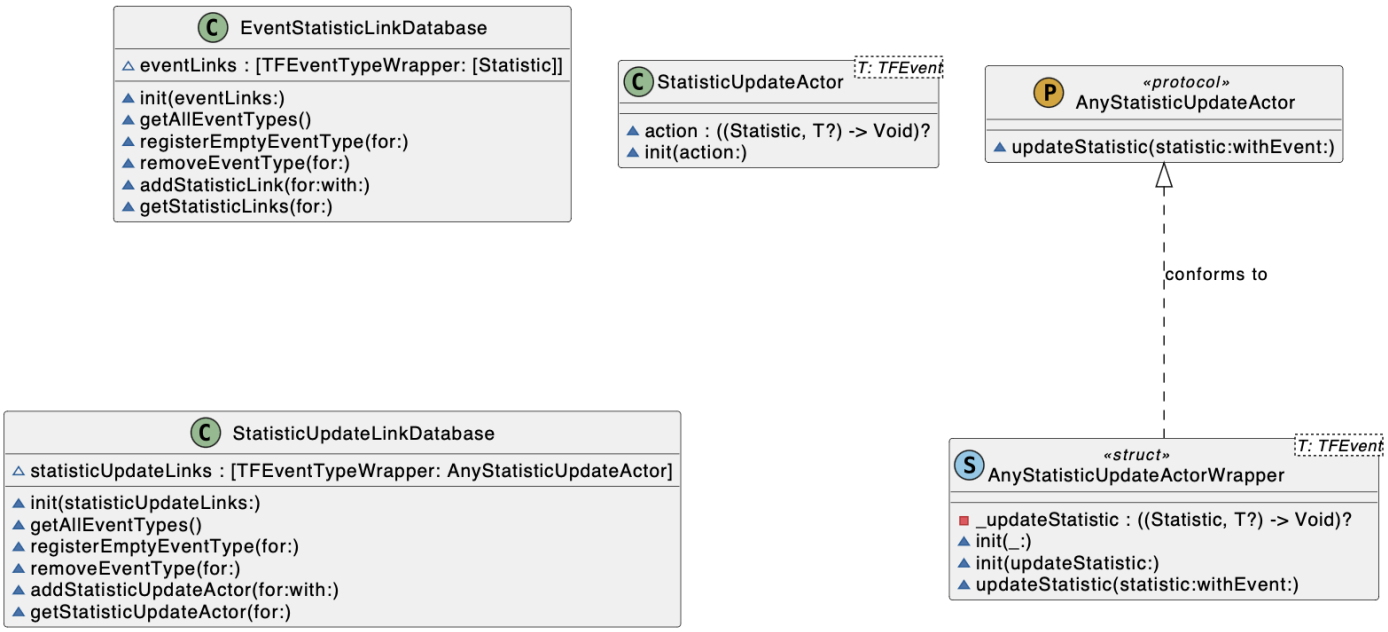
The most basic data structure that encapsulates a **permanent value**, a **current value** (incurred during the course of the game) and a **maximum current value** that shows the maximum current value incurred within any game. Permanent value is utilized by the **AchievementsEngine** and maximum current value is utilized by the **MissionsEngine**.



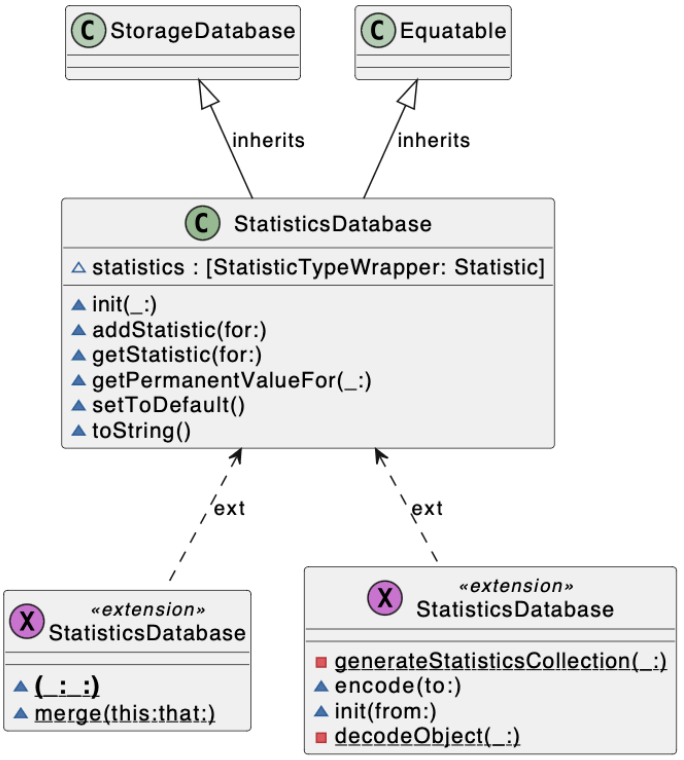
EventStatisticLinkDatabase: Contains a dictionary whose keys are event types and whose values are an array of Statistics which correspond to that particular event type.

StatisticUpdateActor: Contains a closure that takes in a Statistic and an Event and can be executed to update that Statistics. AnyStatisticsUpdateActor and the corresponding wrapper encapsulate the aforementioned closure in a type-erased way such that they can be stored within the StatisticUpdateLinkDatabase.

StatisticUpdateLinkDatabase: Contains a dictionary whose keys are event types and whose values are StatisticUpdateActors.

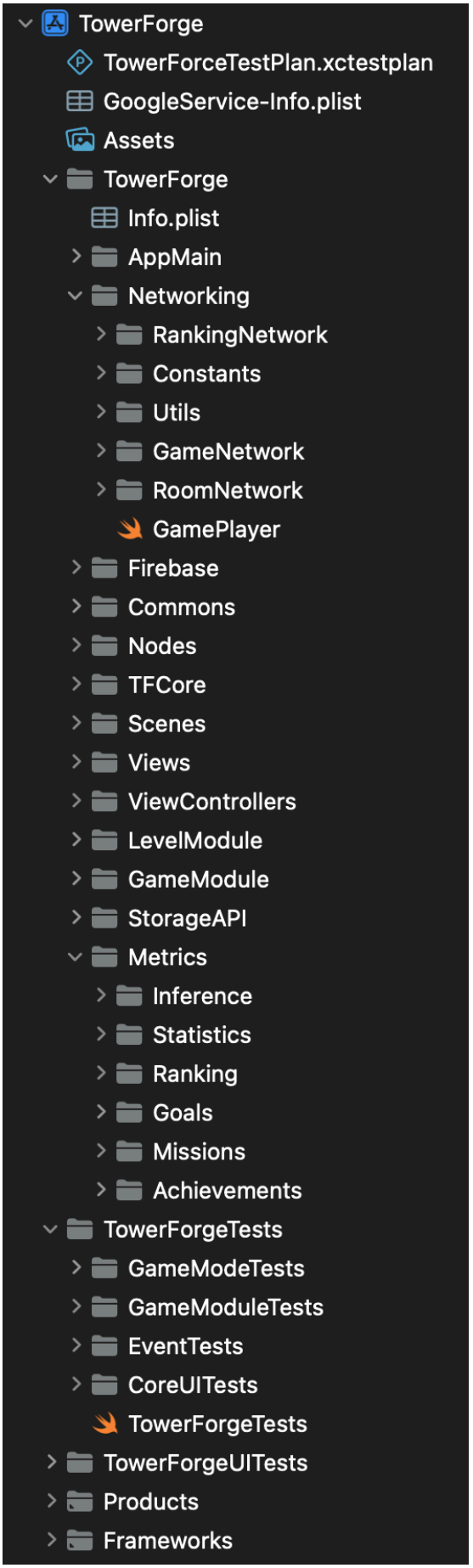


StatisticsDatabase: Contains a dictionary whose keys are statistics types, and whose values are the statistics themselves. This is the core data structure that encapsulates the information contained within each Statistic at run time.



File Structure

We've organized the project as shown on the right. The TowerForge root folder contains the majority of the project file. Below are some files and groups explained (including those not included in the above module diagram).



TestPlan

The TowerForgeTestPlan file is the custom test plan we use, which allows us to have more control over the tests executed. This is particularly important because we use continuous integration to build and test our project using GitHub Actions, and having UITests run for every PR is very resource intensive and time intensive. Thus, we can use this custom TestPlan to disable UITests in the cloud while being able to run them locally when needed.

AppMain

This group contains files related to the Application and App lifecycle, such as the AppDelegate class and storyboard files.

Protocols

This group contains protocols used universally across TowerForge, such as **Animatable** and **Spawnable**.

TFCore

This group contains base elements needed for GUI rendering, including some SpriteKit components.

LevelModule

This group contains files and classes responsible for the majority of game logic, including entities, components, systems, all the respective managers, and level-specific features such as the grid and unit generators.

GameModule

This group contains the GameWorld class which is responsible for the majority of the Game Logic, and other classes needed for game functionality

TowerForgeTests

This group contains unit tests and integrations tests.

TowerForgeUITests

This group contains UI tests.

7 – Reflection

Lessons

ECS is a very powerful design pattern for software that requires very customisable behavior. With **ECS**, in order to add new behavior to an entity, all that needs to be done is to create a new component to hold the data representation relating to the behavior and add a new system to update the components to reflect the expected behavior. This makes **ECS** very suitable for games where there are many different entities that might share various behaviors but have some differing behaviors. As such, it is hard to represent the relationship between the differing entities with just regular **Polymorphism**.

Since we are building a game that belongs to the Tower Defense genre, we decided to make use of the benefits of both **Polymorphism** and **ECS**. Since our game's entities can be clearly defined into 3 types with distinct behavior, we implemented the base class for each of these types. They are **BaseUnit**, **BaseTower** and **BaseProjectile**. **BaseTower** has a health and a position, **BaseUnit** is similar to **BaseTower** but it can also move and **BaseProjectile** does not have health but can move. By defining these base classes, we need not repeatedly write the same code for classes that clearly subclass these base classes while still being able to customize each of their behavior to the nature of **ECS**.

Evaluation

Working with Xcode

One thing that we had greatly underestimated was the issues that can arise from working on an Xcode project. In most other development environments, modified files are tracked directly and singularly, merged conflicts can be handled on a file-by-file basis. In Xcode however, new files are indexed within the .pbxproj file, where each file and the entire file hierarchy is represented in text, alongside the file itself within the project. This presented a major problem when we had to restructure the project's file structure, and that caused merge conflicts that could not be resolved at all. The merge conflict had been caused by the .pbxproj file which was modified due to changes in the folder structure. The corrupted .pbxproj file meant that the conflicting changes had to be manually added again. To avoid this issue in the future, we refrain from changing the project's file structure frequently and when we needed to do so, we ensure that no one had any changes on any existing branches.

ECS & OOP

Initially based on the project proposal period, we had planned to develop the game using a traditional **OOP** design, where entities were represented as classes with predefined properties and behaviors. The decision to adopt the Entity-Component-System (**ECS**) architecture in TowerForge stemmed from our group's recognition of its numerous advantages over a purely Object-Oriented Programming (**OOP**) approach.

By combining **OOP** and **ECS**, TowerForge achieved a balance between object-oriented design and data-driven architecture. **OOP** provided a familiar and intuitive way to define game entities and their behaviors, while ECS offered greater flexibility and scalability by separating data from behavior. This hybrid approach enabled developers to leverage the strengths of both paradigms, resulting in a well-structured, modular, and efficient game architecture.

The adoption of **ECS** required some adjustments to our development workflow and mindset, but ultimately, it proved to be a valuable decision. It allowed us to better manage entity behavior and interactions, reduce complexity, and facilitate the addition of new features with minimal disruption to existing code.

Overall Reflection

Tell me how you think you're doing so far. Are you able to meet your scheduled deadlines? Have you slipped? What did you do (or what do you plan to do) about the slippage?

We believe that we are doing quite well thus far, being able to keep up with our internal deadlines and our forecasted schedule for the project. There were some minor slippages here and there particularly when multiple people work on a single component or dependency, this would naturally introduce some bottlenecks as certain components have to be finished first before progress on the dependent component can be made. One example was that the AIComponent required Systems to be implemented first, which had not yet been implemented by the time AIComponent was completed. However, this also allowed us to rethink our design to be more compatible with SWE principles - we had to ensure that the components we implemented depended as little on other parts of the codebase as possible. In doing so, not only were we able to make progress more effectively, we also learned to appreciate the importance and motivation behind some SWE principles by applying them first hand.

If you don't think you're doing well, what went wrong? What are the problems that your group is currently facing? How do you plan to deal with these problems?

As most of us are new to iOS development, we are also new to working with the Xcode integrated development environment. While we had worked with it individually during the individual project in the first half of the semester, this project is a collaborative effort and multiple people working on the same Xcode project presented its own issues (highlighted earlier). However, with proper communication, we were able to sort out all issues that we might have had, ensuring a seamless dev experience for all team members.

What have you learnt in the last six weeks? How are you going to apply that newfound knowledge to the rest of the project?

We had learned the importance of transparent communication and keeping to a schedule, ensuring that everyone is aware of what everyone else is working on and ensuring that we are all on the same page. This not only allows us to figure out any issues that might arise in the course of us working on TowerForge, but also ensures that everyone has a pleasant experience working together.

Known Bugs and Unresolved Issues

Networking Latency

There might be latency issues due to delayed updates or occasional inconsistencies in the gameplay where actions performed by players may not synchronize seamlessly. This delay might be due to the latency in the multiplayer mode, stemming from a synchronization delay with the Firebase Realtime Database. This inherent challenges deals with handling rapid and frequent data updates in real-time multiplayer scenarios. Latency in data transmission can impede the timely propagation of updates. To mitigate this issue, we can enhance the aspects of the multiplayer infrastructure, we can deliver a smoother and more seamless gaming experience for the players.

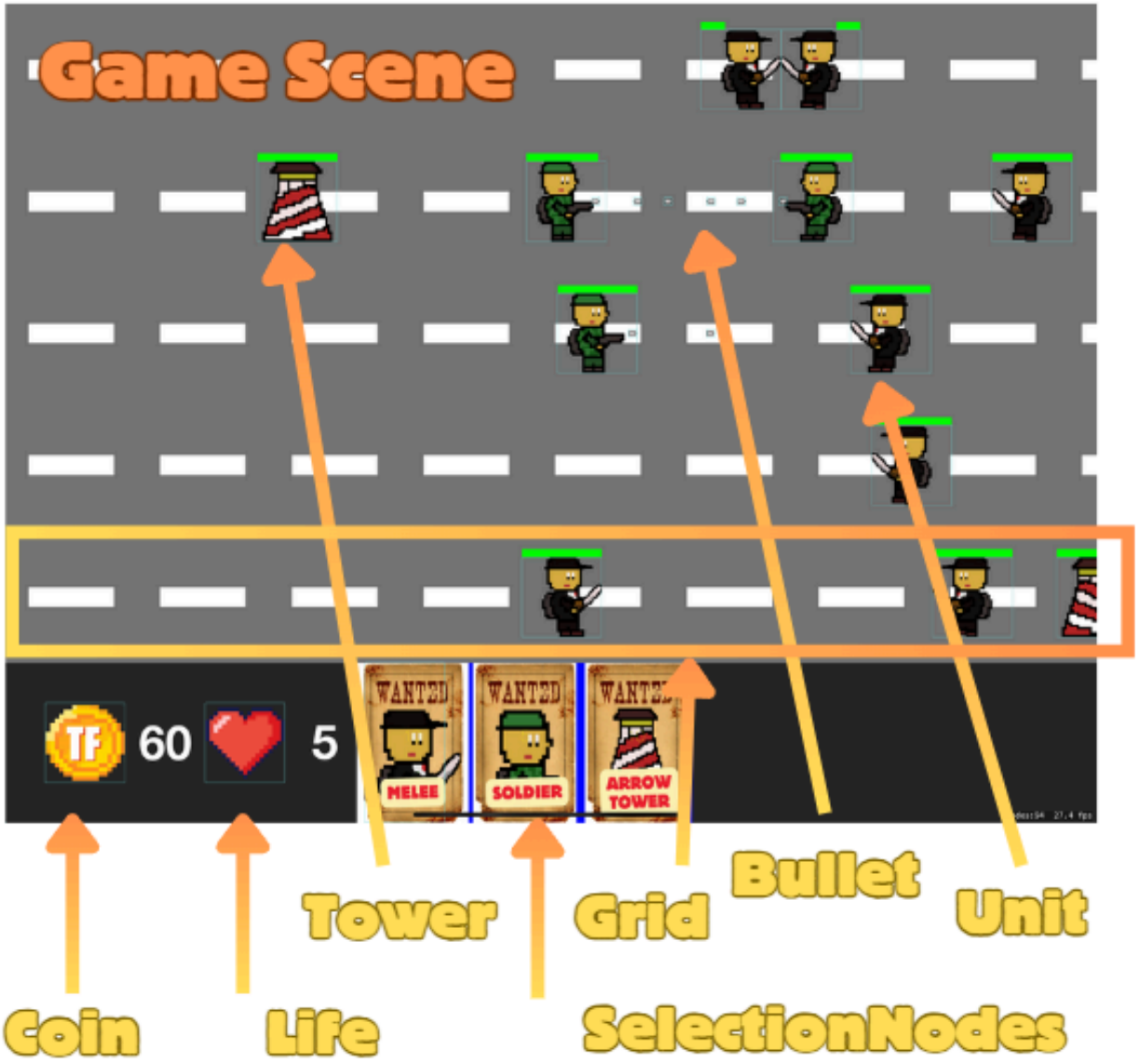
This delay might grant the host player a huge advantage in reaction times, thus to make things slightly more fair, in multiplayer mode both players subscribe to the network for game events so both players will experience similar latency. However, there are still several factors that we cannot control such as internet speeds, distance to the firebase server and more.

Lag Propagation

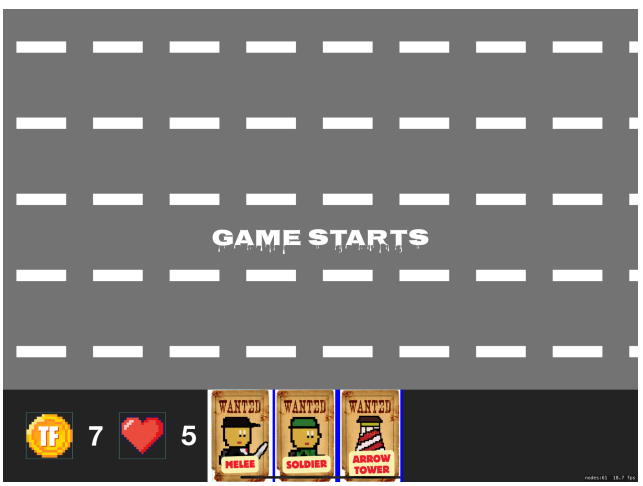
Another issue we encountered was the propagation of lag in multiplayer mode. In order to synchronise the game state of both players, one of the players is designated as the host player and the game state of the host is mirrored onto the other player's game through the game network. However, because of this, if there are issues with the host's set up resulting in their game to lag, it can cause the other player to experience lag in their game as well. For example, all damage is calculated at the host. However, if the host is unable to keep up with the processing, the health of the entities will not change on the client player's side. One way we can fix this is to set up dedicated servers to simulate the games just like the host is currently doing and make both players subscribe to changes in the servers.

8 – Appendix

A – GUI







B – Test Cases

Some of the test cases are implemented and some are designed and planned here.

Startup:

- Upon startup, the user should be redirected to the main screen where they can see a button “Play Game”
- Clicking on “Play Game” should redirect the user to the game screen

Game Screen:

- The user should see their available coins and life points on the bottom-left corner of the screen
- The user should be able to select between the different units (Melee, Soldier, Wizard, Arrow Tower)

Spawning Units:

- With a unit selected, the user should be able to click on the screen to spawn the selected unit on a lane
- Upon spawning a unit, the available coins should decrease by the cost of the unit

Combat:

- All units should stop on contact with an enemy unit on the same lane
- Unit health points should decrease upon being attacked by an enemy unit
- When a unit’s health reaches zero, the unit should be removed from the game screen

Unit Behavior:

- Soldier units should begin shooting bullets when they are in range of an enemy unit on the same lane
- Melee units should attack on contact with an enemy unit
- Arrow Towers should block enemy units from advancing

Power Ups:

- Invulnerability:
    - While in a game, spawn units to fight against the enemy units
    - When the units and enemy units collide and begin to deal damage to each other, activate the invulnerability power up by clicking the shield icon
    - Verify that the player’s units are not receiving any damage for 5 seconds
    - Click on the power up again, verify that the power up is on cooldown for the next 10 seconds
    - Once the cooldown is over, repeat the earlier steps to verify that the power up works as expected
- Damage:
    - While in a game, spawn units to fight against the enemy units
    - When the units and enemy units collide and begin to deal damage to each other, activate the damage power up by clicking the sword icon
    - Verify that the player’s units are dealing extra damage to the enemy’s units for 5 seconds
    - Click on the power up again, verify that the power up is on cooldown for the next 10 seconds
    - Once the cooldown is over, repeat the earlier steps to verify that the power up works as expected
- No Cost:
    - While in a game, activate the no cost power up by clicking the coin icon
    - Select a unit and click on the screen to spawn the unit
    - Verify that the units do not reduce the coins available for 3 seconds
    - Click on the power up again, verify that the power up is on cooldown for the next 30 seconds

- Once the cooldown is over, repeat the earlier steps to verify that the power up works as expected

Unit Behavior:

- Soldier units should begin shooting bullets when they are in range of an enemy unit on the same lane
- Melee units should attack on contact with an enemy uni

Unit Tests

Unit tests have been written for **TFEntity**, **TFComponent**, **EntityManager**, **SystemManager** and **EventManager, Game Modes**.

GameMode

1. Initialization of the game mode
    a. Purpose: Ensures that the **GameMode** instance is initialized correctly with the expected initial properties and state
2. updateGame()
    a. Purpose: Verify that the game state is updated correctly based on the current conditions during the game play
        i. Scenario: Game timer runs out
            1. Expected: Game state should be set to a result based on the game conditions like (WIN, LOSE, DRAW)
        ii. Scenario: Game being paused
            1. Expected: GameMode properties that should be updated gets stopped

GameModeFactory

1. createGameMode()
    a. Purpose: Verify that the **createGameMode** function handles all possible cases defined in the **Mode** enum and returns the game mode instance that is supposedly the right subclass of the **GameMode**

GameRoom

1. Initializing the **GameRoom** class
    a. Purpose: Ensure that the **GameRoom** instance is initialized correctly and returns nil if the room is not available
        i. Scenario: Room is available
            1. Expected: Returns a GameRoom instance
        ii. Scenario: Room is not available
            1. Expected: Returns nil and completion is set to false because it has failed to initialize a gameRoom
2. joinRoom()
    a. Purpose: Verify the functionality of the **joinRoom** method when adding players to the room
        i. Scenario: Attempting to join a full room
            1. Expected: The player should not be added to the room and the completion handler should return false
        ii. Scenario: Attempting to join a non-full room
            1. Expected: The player should be successfully added to the room as either the playerOne or playerTwo.
3. leaveRoom()
    a. Purpose: Verify the functionality of the **leaveRoom** method in removing players from the room
        i. Scenario: Player leaves a room with two players
            1. Expected: the player should be removed from the room.
        ii. Scenario: Player fails to leave the room
            1. Expected: the completion handler should be returning false to indicate unsuccessful leaving of the room
4. isRoomFull()
    a. Purpose: Ensures that it correctly determines if the room is full
        i. Scenario: Check if a room with two players is considered full.

            1. Expected: The method should return true.
        ii. Scenario: Check if a room with one player is considered full.
            1. Expected: The method should return false.
5. makeRoomListener() + makeDeletionRoomListener()
    a. Purpose: Ensure that the room listener is called when the initializing is successfully done.

Authentication Tests

1. Initialisation of the authentication provider
    a. Purpose: Ensures that the AuthenticationProvider instance is initialized correctly with the expected initial properties and dependencies
        i. Expected : AuthenticationProvider instance is initialized successfully, and authentication manager is set.
2. login()
    a. Scenario: User logs in with valid credentials
        i. Expected: Completion block returns authentication data with user ID, email, and username.
    b. Scenario: User logs in with invalid credentials
        i. Expected: Error thrown
3. logout()
4.
    a. Scenario: User logs out
        i. Expected: Completion block returns nil error, indicating successful logout.
5. register()
    a. Scenario: User registers in with valid credentials
        i. Expected: Completion block returns authentication data with user ID, email, and username.
    b. Scenario: User registers in with invalid credentials
        i. Expected: Error thrown
6. getUserDetails()
    a. Scenario: User is logged in
        i. Expected: Completion block returns authentication data with user ID, email, and username.

Game Ranking + LeaderBoard Tests

1. Initialisation tests
    b. Purpose: Ensure that the **GameRankProvider** instance initialized correctly with the expected properties and configuration.
    c. Steps:
        i. Instantiate a `GameRankProvider` object.
        ii. Verify that the Firebase Realtime Database reference is correctly initialized and points to the appropriate location.
2. setNewRank()
    a. Purpose: Verify that the setNewRank method correctly updates the leaderboard with a new rank if the player is new or the rank is higher than the current player's score
    b. Steps
        i. Create a mock GameRankData object representing a new rank.
        ii. Call the `setNewRank` method with the mock rank data.
        iii. Check that the leaderboard data in Firebase is updated accordingly.
        iv. Verify that the method returns successfully without errors.
3. getTopRanks()

a. Purpose: Verify that the `getTopRanks` method retrieves the top ranks from the leaderboard in the correct order.
b. Steps
    i. Call the `getTopRanks` method with a specified sorting order (increasing or decreasing).
    ii. Check that the returned list of ranks is sorted according to the specified sorting order.
    iii. Verify that the method returns the expected number of top ranks without errors.

4. getHighScore()
    d. Purpose: Ensure that the getHighScore method retrieves the highest score for a specific player from the leaderboard.
    e. Steps:
        i. Provide a known player ID for an existing player in the leaderboard.
        ii. Call the `getHighScore` method with the player ID.
        iii. Verify that the method returns the highest score for the player without errors.

Integration Testing

Authentication Integration Tests

1. Login and Logout
    a. Scenario: User logs in and then logs out
    b. Steps:
    1. User logs in with valid credentials.
    2. After successful login, the user's authentication data is retrieved.
    3. User's authentication status is confirmed as logged in.
    4. User logs out.
    5. After successful logout, the user's authentication status is confirmed as logged out.

LeaderBoard Integration Tests

1. Set New Rank and Retrieve Top Ranks
    a. Purpose : Ensure that setting a new rank updates the leaderboard correctly and retrieving the top ranks reflects the updated state.
    c. Scenario: User logs in and then logs out
    d. Steps:
        a. Set a new rank using the setNewRank method with a new player and a new score. The score needs to be higher than whatever is in the database
        b. Retrieve the top ranks using the getTopRanks method.
        c. Verify that the new rank is included in the list of top ranks and that the list is sorted accordingly.
        d. Confirm that the leaderboard data is updated in Firebase and reflects the changes made.

Game Room Integration Tests

1. Room change notification for **GameRoomDelegate**
    a. Purpose: Ensure that the delegate receives notifications for room changes
        i. Scenario: Add a player in the database
            1. Expected: The GameRoom listens for changes and notifies using the GameRoomDelegate to send changes
2. Room Finding

    a. Purpose: Validate the integration between the findRoomWithName method and database retrieval.
    b. Steps:
        i. Create a room in the database with a specific name.
        ii. Use the findRoomWithName method to retrieve the room.
        iii. Verify that the retrieved room matches the expected room state.
3. Room Deletion
    a. Purpose: Validate the integration between the leaveRoom method and room deletion from the database.
    b. Steps:
        i. Create a room instance.
        ii. Join the room with multiple players.
        iii. Leave the room for each player until the room becomes empty.
        iv. Verify that the room is deleted from the database.
4. Joining and Leaving Room
    a. Purpose: Verify the integration between the joinRoom and leaveRoom methods along with the database operations.
    b. Steps:
        i. Create a room instance.
        ii. Join the room with multiple players.
        iii. Ensure that players are correctly added to the room and the database.
        iv. Leave the room for each player.
5. Synchronised Starting of Game
    a. Purpose: Ensures that both players start at roughly the same time so that no player has the advantage
    b. Steps:
        i. Create a room instance
        ii. Join the room with multiple players.
        iii. Press start with all players
        iv. Game should start for both players at the same time with a small margin of error
6. Host Transfer
    a. Purpose: Ensures that there is always a host in the game room.
    b. Steps:
        i. Create a room instance. This player will be the current host. Verify by attempting to change the GameMode.
        ii. Join the room with another player.
        iii. Leave the room with the host player
        iv. Host position should be transferred to another player. Verify by trying to change the GameMode.
7. GameMode Selection
    a. Purpose: Verify that GameMode changes and game starts at selected GameMode for both players
    b. Steps:
        i. Create a room instance.
        ii. Join the room with another player
        iii. Use the host player to change the GameMode to Capture The Flag
        iv. Verify that the GameMode listed on client player is also Capture The Flag
        v. Start the game for both players.
        vi. Verify that the GameMode of game is Capture The Flag
        vii. Repeat for Death Match Mode

Game Network Integration Tests

1. Spawning
    a. Purpose: Verify that any unit spawns by one player shows in the other player's game
    b. Steps:
        i. Start a multiplayer game
        ii. Spawn units with one player
        iii. Pan the camera with the other player and verify that the unit spawns
        iv. Repeat steps iii and iv with other player
2. Damage
    a. Purpose: Verify that any damage in host game is reflected in client game
    b. Steps:
        i. Start a multiplayer game
        ii. Spawn units on same lane with both players
        iii. Let the entities attack each other
        iv. Verify that any health reduction in entities in host game is reflected in client game
3. Removal
    a. Purpose: Verify that any removal of entities in host game is reflected in client game
    b. Steps:
        i. Start a multiplayer game
        ii. Spawn unit on the same lane with both players
        iii. Let the entities attack each other and wait for an entity to die and be removed from host game
        iv. Verify that the same entity has been removed in the client game
4. Concede
    a. Purpose: Verify that conceding by one player will result in other player winning
    b. Steps:
        i. Start a multiplayer game
        ii. Concede the game with one of the players
        iii. Concede player will be shown the losing screen and the other player will be shown the winning screen
5. Life
    a. Purpose: Verify that any damage to player base in the host game will be reflected in client game
    b. Steps:
        i. Start a multiplayer game in Capture The Flag mode
        ii. Spawn an entity with one of the players and let it make its way to the other player's base
        iii. Verify that a life has been removed for the player whose base has been damaged on the host game
        iv. Verify that a life has been removed for the same player in the client game
        v. Repeat steps iii and iv for other direction

6. Death
    a. Purpose: Verify that any unit kills accumulated in the host game will be reflected in the client game
    b. Steps:
        i. Start a multiplayer game in DeathMatch mode
        ii. Spawn units on the same lane with both players
        iii. Let the entities attack each other and wait for an entity to die and be removed
        iv. Verify that the player who killed the entity has their kill count increase in the host game
        v. Verify that the same player has their kill count increased in the client game
7. Powerup
    a. Purpose: Verify that power ups will take effect on both player's game state.
    b. Steps:
        i. Start a multiplayer game
        ii. Spawn units on the same lane with both player
        iii. Wait for entities to come into contact and damage each other
        iv. Activate invulnerability with one of the players
        v. Validate that the entities of the player who activated invulnerability is no longer receiving damage on both host and client game state
        vi. Wait for invulnerability powerup to end
        vii. Validate that the same entities are now taking damage
        viii. Repeat iv to vii for other player

Storage + Sync Tests

1. Initial boot
    a. Starting the game should initialize an empty statistics database file and metadata
2. Login to new account
    a. Logging in with existing progress to a new account should upload progress to the FirebaseURL
3. Login to exist account
    a. Immediately logging in to existing account should replicate user stats locally as they are in the cloud
    b. Logging in after generating stats locally should merge local stats with stats in the cloud

UI Testing

Authentication UI Tests

1. Login and Logout UI button
    a. Steps:
        i. Launch the application and navigate to the authentication screen.
        ii. Verify that the login button is visible.
        iii. Tap on the login button to navigate to the login screen.
        iv. Enter valid login credentials (e.g., email and password).
        v. Tap on the login button.
        vi. Verify that the login is successful and the user is redirected to the home screen.
        vii. Verify that the login button has now changed to a logout button.
        viii. Tap on the logout button.
        ix. Verify that the logout is successful and the user is redirected to the authentication screen.
        x. Verify that the login button is visible again.
        xi. Verify that the global ranking button is now visible on the home screen.

C – Project Schedule

Sprint	Week	Date	Task
-	Week 7	March 3	Admin Tasks - Set up Workflow, CI/CD, etc
1	Week 8	March 10	Models (Grid, Game Entities, Persistence) + Unit test
	Week 9	March 17	Game Loop, Event Queue, Game Engine, UI + Controllers
2	Week 10	March 24	Behaviour Customisation (Types, Paths, Enemy Prioritisation)
	Week 11	March 31	Multiplayer Network Adapter* (or other extensions)
3	Week 12	April 7	Multiplayer Network Adapter* (or other extensions)
	Week 13	April 14	Finalize project and Bug Fixing

Sprint Objectives

Sprint 1 (March 10 - March 23)

- We want to have the basic features and have a robust skeleton of the game which will be easily extendable.
- We want to have rigorous unit testing for all the components that we implement.
- We want to have code coverage analysis and continuous integration workflows set up for the rest of the project.

Sprint 2 (March 24 - April 6)

- We want to have basic customisation and possibly implement one of the extensions.
- Developing and integrating into a working game.
- We want to begin integration testing for the features that we have implemented and set up a workflow to continuously test new features

Sprint 3 (April 7 – April 17 STePS)

- We want to include the bells and whistles + other extensions that would be viable and effective to be added to the game and also polish up the game for final distribution/release.
- Finalizing the game
- Closer to the deadline we would want to implement a feature freeze and work on perfecting the features that we already have, sorting out any possible issues.

D – Individual roles and contribution

Week 8 Todos

Task	Assignee	Status
TF Adaptors	Vanessa	Done
Components + TFComponents	Vanessa	Done
Entities + EntityManager + TFEntity	Zheng Ze	Done
Grid class	Keith	Done
Systems + TFSystem	Rubesh	Done
CI/CD + SwiftLint + Github setup	Rubesh	Done

Week 9 Todos

Task	Assignee	Status
Game Engine	Rubesh	Done (Integrated with GameWorld)
Event Queue + Event Manager + Events	Zheng Ze	Done
GameScene + Game Controller	Vanessa	Done
Enemy Spawner	Vanessa	Done
Collision	Zheng Ze	Done (Using SpriteKit's collision)
UI + Nodes + Sprites	Vanessa	Done
Renderer	Zheng Ze	Done
GameWorld	Zheng Ze	Done

Week 10 + 11 Todos

Task	Assignee	Status
Missions / Achievements	Rubesh	In progress
Game Modes	Vanessa	Done
Storage (Base Implementation with FileManager)	Rubesh	Done
Power Ups + Spells	Rubesh + Keith	Done – to add more power ups
Render Pipeline	Zheng Ze	Done
Event Pipeline	Keith	Done
Contact System	Zheng Ze	Done
Camera Panning	Zheng Ze	Done
Player base and life reduction	Vanessa	Done
Remove out of bounds entities	Vanessa	Done
Spawn Location Restrictions	Zheng Ze	Done
Firebase Adapters	Vanessa	Done
RoomNetwork	Vanessa	Done
GameNetwork	Zheng Ze	Done
Game States + UI	Keith, Vanessa	Done
Sound Manager + Sound Handling	Rubesh	Done

Week 12 Todos

Task	Assignee	Status
Ready System + Synchronised Start	Vanessa	Done
More RemoteEvents to reflect status of host game	Zheng Ze	Done
Leaderboard	Vanessa	Done
Firebase Authentication Provider	Vanessa	Done
Game Rank Data	Vanessa	Done
Storage Archi + Auth Handler	Rubesh	Done
Local+Remote Dual Storage Sync	Rubesh	Done
Storage Conflict Resolution	Rubesh	Done
Cross-Platform Player Progress Saving	Rubesh	Done
MetricsAPI Architecture (Statistics)	Rubesh	Done
AchievementsEngine (MetricsAPI)	Rubesh	Done
MissionsEnging (MetricsAPI)	Rubesh	Done
RankingEngine (MetricsAPI)	Rubesh	Done
Player Stats Screen (i.e. Achievements List, Mission List & Rank Showcase)	Rubesh	Done
More Power Ups	Keith	Done
Migrate Movement to MoveEvent	Keith	Done
Optimize Rendering Pipeline	Zheng Ze	Done
Fix sprite layering	Zheng Ze	Done
Fix UI resolution issue	Zheng Ze	Done
Multiplayer Gamemode Selection	Zheng Ze	Done
Fix Touch Sensitivity	Zheng Ze	Done